

RETI E SISTEMI OPERATIVI

Introduzione alle reti e livello di applicazione:

Protocollo:

Un protocollo definisce il formato, l'ordine dei messaggi inviati e ricevuti in una rete tra le entità e le azioni intraprese su questi messaggi.

Standard:

Documenti emanati da degli enti di standardizzazione, uno dei più famosi è IETF (internet engineering task force).

Internet as Service: è un'infrastruttura che fornisce dei servizi di connettività per delle applicazioni.

Invio di pacchetti di dati: un messaggio non viene direttamente inviato:

- Viene diviso in pacchetti di **L** bit.
- Trasmessi in una rete di banda **R**.

Il tempo necessario al trasferimento è detto **delay di trasferimento** ed equivale a $\frac{L}{R}$.

Nucleo della rete: insieme di router interconnessi che compiono due azioni:

- **Forwarding:** azione di inoltrare di un pacchetto verso un nodo successivo.
- **Routing:** instradamento, permette di costruire il percorso tra la sorgente e la destinazione.

Packet Switching: divisione dei pacchetti e indirizzamento attraverso vari router fino al destinatario:

- **Store and Forward:** tutto il pacchetto deve essere ricevuto dal router prima che possa essere avviato il trasferimento successivo.
- **Queuing:** Quando c'è un maggior carico in input rispetto alla velocità di trasferimento (transmission rate) i pacchetti vengono messi in coda in attesa di essere trasmessi. Se la coda non è in grado di gestire una grande quantità di pacchetti (limiti di memoria) questi vengono persi.

Circuit Switching: il Circuit switching (reti a commutazione di circuito rappresentano un'alternativa al packet switching). Quando due dispositivi desiderano comunicare, viene stabilito un percorso dedicato attraverso la rete. Questo percorso rimane esclusivamente dedicato a quella specifica connessione per l'intera durata della sessione.

- I circuit switching risultano migliori per una trasmissione di dati rivolti alla condivisione delle risorse, sebbene presentino problemi di coesistenza della rete.

Struttura di internet (rete di reti):

Internet ha una struttura gerarchica costituita da:

- Connessioni tra **HOST** e **IPS** (internet service provider).
- Interconnessione tra i vari ISP.

La connessione diretta tra internet service provider non è realizzabile; pertanto, ogni ISP si connette a ISP globali che a loro volta sono collegati tramite degli IXP (internet Exchange point).

Agli ISP globali sono connessi gli ISP regionali che si fanno carico di una determinata zona geografica, attraverso questa rete i Content Provider Network offrono servizi agli utenti finali.

Tipologie di ritardo in una rete a commutazione di pacchetto:

- **Ritardo di elaborazione:** viene introdotto da un commutatore per svolgere tutte le operazioni necessarie e procedere all'inoltro del pacchetto. Quando il pacchetto entra in un commutatore quest'ultimo deve svolgere due operazioni. Controllare che non ci siano errori nel pacchetto e andare a cercare nella tabella di inoltro qual è il link di uscita su quale il pacchetto deve essere inviato. I ritardi di operazione sono ritardi bassi, nell'ordine dei nanosecondi.
- **Ritardo di accodamento:** è un tipo di ritardo diverso per ogni singolo pacchetto che transita in rete. In situazioni standard rientrano nell'ordine dei microsecondi/millisecondi.
- **Ritardo di trasmissione:** è il tempo necessario da quando viene inviato il primo bit sul canale a quando viene ricevuto l'ultimo bit del pacchetto.
- **Ritardo di propagazione:** è un ritardo che dipende dal mezzo adottato e si verifica per la necessità di un pacchetto di andare da un punto A ad un punto B. Si calcola come $\frac{D}{S}$ dove D è la lunghezza del link fisico e S è la velocità di propagazione del link.

Ritardo di accodamento:

- $\vec{a}$ average packet arrival rate.
- L packet length (bits).
- R link bandwidth (bit transmission rate).

$$\text{Traffic Intensity} = \frac{L * \vec{a}}{R}$$

- Più il valore è vicino a 0 più il mio rate di servizio è grande rispetto al rate di arrivo dei bit.
- Più il valore è vicino a 1 più il mio rate di servizio è minore rispetto al rate di arrivo dei bit.

Differenza tra ritardo di trasmissione e ritardo di propagazione:

Entrambi i ritardi dipendono dalla tipologia di collegamento, il ritardo di trasmissione non dipende in alcun modo dalla distanza tra due commutatori o dalla lunghezza del link, ma solamente da come utilizzo il mezzo e da come spingo i bit su di esso, oltre un certo limite non mi posso spingere. Mentre per quanto riguarda il ritardo di propagazione dipende dal mezzo, mezzi diversi di lunghezza diversa portano a ritardi diversi.

Ritardo End to End:

Rappresenta il ritardo tra il sistema periferico sorgente e il sistema periferico destinazione, è la somma di tutti i contributi per tutti i commutatori che vengono attraversati a cui vengono aggiunti dei ritardi ulteriori introdotti dai sistemi periferici.

Throughput

È la quantità di bit nell'unità di tempo che vengono inviati dalla sorgente alla destinazione, può essere di due tipi:

- **Istantaneo:** rate in un determinato momento in un dato punto.
- **Medio:** rate lungo un periodo di tempo.

Si calcola trovando il minimo tra le velocità di trasferimento nel collegamento interessato.

Bottleneck:

Sul percorso tra la sorgente e la destinazione esiste un collegamento, collo di bottiglia (bottleneck) che va a vincolare quello che è il throughput end to end. È vero che esistono collegamenti che sono colli di bottiglia, tutto dipende da come viene usata la rete, la rete deve essere costruita in modo tale che non si verifichino bottleneck nella rete di core; pertanto, si ricorre all'**overprovisioning** che consiste nell'avere dei link nelle reti di core molto maggiori dei link nelle reti di accesso.

Architettura a strati (di Internet):

Internet è diviso in layers, ogni layer implementa un servizio basandosi sul layer immediatamente inferiore, se modifico i servizi offerti ad un livello non devo modificare i servizi degli altri livelli.

Layered internet protocol stack: (approccio top-down)

- **Applicazione:** include i protocolli a supporto delle applicazioni distribuite (http, smtp, dns).
- **Trasporto:** abilitare un trasferimento di dati tra processi su macchine diverse.
- **Rete:** ha il compito dell'istadamento far sì che un pacchetto arrivi a destinazione (ip, routing).
- **Collegamento:** si occupa del trasferimento tra due nodi di rete che sono adiacenti.
- **Fisico:** trasferimento dei bit sul cavo.

DNS:

Il DNS (**Domain name system**) è un protocollo di livello applicativo che serve per associare un host name ad un indirizzo IP sfruttando gerarchie di nameserver che sono suddivisi in:

- **Root DNS server:** per ottenere l'indirizzo di un ulteriore server associato al top level domain. Lista di server contenenti correlazioni IP/root domain, sono 13 nel mondo replicati più volte.
- **Top level-domain TLD:** responsabili di domini top level come (.com, .it, .net).
- **DNS di autoritativi:** sono server DNS gestiti da organizzazioni (amazon, yahoo, unimib).
- **DNS main server:** è un server specifico per risolvere qualsiasi indirizzo, può restituire l'indirizzo IP in due modi diversi:
 - Scatenando il sistema DNS.
 - Prendendo l'indirizzo IP associato in una cache che lui mantiene.

DNS name resolution:

Quando devo ottenere la risoluzione dell'host name in un indirizzo IP possono essere adottate due strategie diverse:

- **Iterated query:** è quella di solito viene utilizzata dai server DNS. Il server DNS non recupera la risposta completa alla query, piuttosto fornisce un riferimento ad altri server DNS che potrebbero avere la risposta.
- **Recursive query:** io stesso come name server contatto e aspetto una risposta dal DNS server più in basso nella gerarchia.

Livello di trasporto:

Il livello di trasporto fornisce una comunicazione logica tra processi applicativi eseguiti tra host diversi. Una comunicazione logica permette ai processi di comunicare come se fossero sullo stesso host. Le azioni intraprese in questo livello sono:

- **Sender:** prendere i messaggi dall'applicazione crearne dei segmenti e passarli al livello di reti.

- **Receiver:** riassembla i segmenti e li passa al livello applicativo.

I maggiori protocolli a livello di trasporto sono **UDP** e **TCP**.

Socket:

Quando un processo deve inviare o ricevere messaggi tramite livelli di trasporto si affida al socket, interfaccia attraverso cui vengono spinti i messaggi verso il livello di trasporto.

- **Inviante:** riceve un messaggio dall'applicazione, determina i valori dell'header, crea il segmento e lo passa a livello di IP.
- **Ricevente:** riceve dal livello IP, controlla l'header, estrae il messaggio e demultiplexa tramite il socket.

Protocollo TCP:

Il protocollo TCP (transmission control protocol) viene utilizzato in applicazioni elastiche, **(applicazioni non tolleranti della perdita ma sono tolleranti del ritardo)**, infatti garantisce che tutti i segmenti vengano consegnati al destinatario in ordine. Inoltre, TCP è un protocollo connection oriented (orientato alla connessione) perciò richiede che vengano stabilite delle connessioni tra i processi coinvolti.

Ha diverse caratteristiche:

- **Controllo di congestione:** mette in atto delle procedure per le quali se avverte che la rete è congestionata, riduce il rate di invio dei messaggi.
- **Controllo di flusso:** riguarda il destinatario, se il mittente invia tanti segmenti al destinatario e quest'ultimo è in difficoltà a gestire i dati, TCP può abbassare il rate di invio da parte del mittente.

Protocollo UDP:

Il protocollo UDP (user datagram protocol) viene utilizzato in applicazioni real-time non sensibili alla perdita di dati, è una sorta di estensione del protocollo principale di rete (protocollo IP). Il protocollo IP è un protocollo di tipo best effort, non garantisce la consegna ordinata dei messaggi.

Ha diverse caratteristiche:

- **Non c'è controllo sulla congestione**, le congestioni fanno perdere pacchetti.
- Non ha bisogno di connessione, non c'è handshake, garantisce un setup più semplice e un header di minori dimensioni.

Introduce solo le funzionalità minime che devono essere offerte dal livello di trasporto senza implementare nulla di più.

Nessuno dei protocolli UDP e TCP dà garanzie:

- In termini di ritardo.
- garanzie sulla banda, posso ritagliare una banda minima di x megabit al secondo per la mia connessione.

Multiplazione e demultiplazione:

- **Multiplazione:** l'operazione di multiplexing è un'operazione che permette di gestire a livello di trasporto dati inviati da processi differenti. L'operazione di multiplazione viene effettuata inserendo dei valori specifici negli header che permettono di distinguere che i messaggi sono generati da processi diversi e hanno utilizzato socket differenti.

- **Demultiplazione:** permette di andare a smistare tra i diversi socket i segmenti utilizzando le informazioni inserite negli header nella fase di multiplazione.

Connessione:

Può esserci ma anche non esserci

- **Connection oriented:** i socket sono identificati da source IP address, source port number, dest IP address, dest port number in questo modo il ricevente può supportare più socket contemporaneamente (ogni socket è connesso con un client).
- **Connectionless:** la creazione del socket avviene specificando l'IP e la porta di destinazione, quando l'host riceve un segmento lo reindirizza verso la porta specificata.

Trasferimento affidabile dai dati:

Quando abbiamo un protocollo che permette la consegna affidabile tra processi a livello di trasporto, abbiamo dei dati che si affidano ad un canale sicuro. Nella realtà a livello di rete abbiamo un servizio di consegna inaffidabile. La complessità del protocollo dipende dall'inaffidabilità del canale, più il canale è inaffidabile più necessito di un protocollo complesso. Per implementare un meccanismo di consegna affidabile, utilizzo un meccanismo di feedback in cui il destinatario mi comunica se ha ricevuto i dati che ho inviato.

I messaggi specifici di controllo prendono il nome di riscontri o acknowledgments:

- **Acknowledgments** = riscontri positivi = ack.
- **Negative acknowledgments** = nack.

Risoluzione del problema delle perdite:

Per risolvere il problema delle perdite a livello di trasporto si inserisce un timer. Nel momento in cui invio il mio segmento imposto un timer, se entro la scadenza del timer ricevo un ack o un nack sono a posto, se non ricevo nulla ritrasmetto.

Questo è il caso più sofisticato del protocollo **stop-and-wait**. Settare il timer in modo opportuno è fondamentale, se il timer è troppo lungo è necessario aspettare troppo tempo prima di ritrasmettere lo stesso segmento, non utilizzando in modo efficiente il canale di trasmissione. Se accorcio troppo la lunghezza del timer si verificano delle ritrasmissioni inutili. Prima di trasmettere un nuovo segmento devo aver ricevuto l'ack del segmento precedente.

Una regola fondamentale per impostare il timer è porlo almeno oltre il round trip time. Il **round trip time o (RTT)** rappresenta il rapporto tra il tempo impiegato nella trasmissione del segmento utile e il tempo totale trascorso.

Il tempo sul tempo totale che è stato speso per trasmettere informazione utile è detto **utilità** e si ricava nel modo seguente:

$$U = \frac{L/R}{\left(RTT + \frac{L}{R}\right)}$$

Protocolli a finestra scorrevole:

Hanno il compito di migliorare l'efficienza di trasmissione, si possono trasmettere fino a **w** segmenti mentre si aspetta l'ack del primo che è stato inviato. Per garantire una trasmissione continua e ottenere un'utilità del 100% devo fare in modo che il mio ack venga ricevuto prima che abbia finito di inviare i **w** segmenti.

$$W * \frac{L}{R} \geq RTT + \frac{L}{R}$$

Ci sono diversi protocolli a finestra scorrevole:

- **Protocollo GO-BACK-N:** è la soluzione più semplice dal punto di vista implementativo, scorro la mia finestra ogni volta che ricevo un ack, se scatta il time-out ritrasmetto tutti i segmenti dal primo che non è stato riscontrato, se ricevo dei segmenti duplicati dopo una perdita questi vengono scartati. Per mezzo di un protocollo così fatto ho sempre garanzia che andrò a inviare i segmenti in ordine, non ho necessità di un buffer a ricevitore. Un protocollo di questo tipo funziona con un unico timer, ogni qualvolta ottengo un riscontro di un segmento faccio ripartire l'unico timer disponibile.
- **Protocollo SELECTIVE REPEAT:** selective repeat ritrasmette esclusivamente il segmento che è andato perso. Gli ack sono individuali, ogni singolo ack mi riscontra esclusivamente il segmento al quale associato, è necessario avere un timer per ogni segmento e un buffer a ricevitore.

Generalmente protocolli come go-back-n e selective repeat non sono utilizzati nelle reti reali, bensì vengono utilizzati protocolli ibridi che implementano funzioni di entrambi.

Numeri di sequenza:

I numeri di sequenza sono rappresentati da un numero finito di bit, trasportato in un campo dell'header. I numeri di sequenza sono limitati, quindi vengono ripetuti, ciò può creare ambiguità dal momento che si potrebbe verificare che due segmenti inviati dallo stesso host verso la stessa destinazione abbiano lo stesso numero di sequenza ma in realtà siano associati a due segmenti diversi. Questa ambiguità può essere risolta andando ad impostare adeguatamente il valore di w . Se n è massimo numero di sequenza che posso avere, nel caso di go-back-n se imposto $n > w + 1$, mentre per selective repeat $n > 2 * w$.

TCP (overview):

TCP è un protocollo point to point, ovvero coinvolge un mittente e un destinatario, è un protocollo orientato al byte e non al segmento. Permette la trasmissione full duplex (bidirezionale) che è quindi scambi di dati in entrambi le direzioni, introduce l'MSS dimensione massima del segmento. Con TCP vengono implementati dei controlli di congestione e flusso con finestre specifiche, è necessaria una fase di handshaking per andare a creare e stabilire la connessione che inizializza lo stato per mittente e destinatario.

Numeri di sequenza in TCP:

- **Sequence number:** indica il primo byte nel segmento dati.
- **Acknowledgments:** indica il numero di sequenza del byte successivo che ci si aspetta di ricevere dall'altra parte.

TCP round trip time:

Il timer deve essere impostato leggermente più lungo del round trip time in modo tale da non reagire troppo lentamente. Nelle reti reali L'RTT oscilla per ogni segmento inviato, in generale si esegue una media di diversi RTT per diversi segmenti precedenti ignorando le ritrasmissioni. La stima dell'RTT è chiamata *estimatedRTT* e viene definita come media mobile ponderata esponenzialmente.

- **SampleRTT:** tempo trascorso tra la trasmissione di un segmento e il ricevimento di un ack.
- **EstimatedRTT:** $(1 - \alpha) * EstimatedRTT + \alpha * SampleRTT$
- **Timeout interval:** $EstimatedRTT + 4 * DevRTT$

- **DevRTT:** $(1 - \beta) * DevRTT + \beta(SampleRTT - EstimatedRTT)$. È calcolato come EstimatedRTT ma tiene in considerazione la differenza in valore assoluto tra SampleRTT e EstimatedRTT; quindi, tanto più la differenza è grande tra EstimatedRTT e SampleRTT tanto più il mio margine di sicurezza è alto.

TCP fast retransmit:

Con TCP si può ritrasmettere solo quando scade un time-out; tuttavia, in determinate situazioni è possibile ritrasmettere non perchè scade il time-out ma perchè si sono ricevuti dei riscontri negativi. Se ricevo 3 ack duplicati oltre al primo ack, con il meccanismo di **fast retransmit** posso ritrasmettere subito il segmento mancante che ha il numero di sequenza più piccolo, quello più lontano temporalmente che è andato perso.

Flow control:

Controlla la velocità di trasmissione e la imposta di conseguenza, evitando di far andare in overflow il buffer del ricevente. Per farlo, il ricevente restituisce lo spazio libero nel buffer nel campo rwnd.

TCP connection management:

Prima di poter iniziare a inviare dati tra un mittente e un destinatario è necessario effettuare una procedura di handshaking, si va a stipulare un accordo sia per effettuare una connessione sia per stabilire i parametri di tale connessione. Il setup della connessione per TCP è un tre way handshaking.

Chiusura della connessione in TCP:

La chiusura della connessione in TCP può avvenire in 2 modi:

1. **Chiusura graceful:** accordata da entrambi le parti: il server prima di chiedere la connessione invia ulteriori dati mancanti.
2. Il client vuole chiudere, il server non ha alcun messaggio da inviare e si chiude la connessione.

Controllo di congestione:

Quando si verifica una congestione in rete, si ha una situazione per la quale l'utilizzo delle risorse va a saturazione e si ha un throughput end to end che tende al massimo valore possibile. Per via della congestione vado ad avere sempre più ritrasmissione, per recuperare dalle perdite infatti, invio nuovo traffico, il goodput si riduce fino ad arrivare allo zero. Pertanto, si mettono in atto dei meccanismi di riduzione del rate:

- **End to End:** viene effettuato dal sender in modo autonomo.
- **Network-assisted:** le decisioni vengono prese sia end to end sia con l'aiuto di router che mandano dei riscontri sul loro stato di congestione.

Approccio di TCP alla congestione:

- **AIMD (additive increase multiplicative decrease):** il sender aumenta il rate di invio progressivamente fino a che non si verifica una perdita, momento in cui il sending rate si riduce in maniera drastica e si ricomincia a farlo salire. Il sending rate viene aumentato di un MSS ogni volta che si ha un RTT nuovo, quando si verifica la perdita il sending rate viene dimezzato.

Il controllo di congestione e il controllo di flusso coesistono nel sender. Per definire la finestra di trasmissione consideriamo il minimo tra la finestra di congestione e la finestra di ricezione.

La finestra di congestione in TCP viene impostata tramite tre fasi:

1. **Slow start:** quando si inizia con la connessione la finestra di gestione aumenta di dimensione in modo esponenziale, a un certo punto si raggiunge una crescita lineare, da quel punto in poi inizia la fase di congestion avoidance.
2. **Congestion avoidance:** lo switch avviene quando la finestra di gestione vale un $1/2$ rispetto al valore prima dell'ultimo time-out. Si inizializza una variabile denominata **ssthresh** che viene resa la metà del valore della finestra di gestione ogni qualvolta si verifica un evento di perdita.
Un evento di perdita è strettamente legato alla versione di TCP, nel caso di TCP reno dipende dal fatto che il mio evento di perdita sia legato a 3 ack duplicati o alla scadenza di un timer.
3. **Fast recovery (valida solo se si ha una fast retransmit):** se l'evento di perdita è associato a 3 ack duplicati, dimezzo la ssthresh e imposto la mia congestion window a ssthresh + 3MSS perchè ho avuto 3 ack duplicati. Comincio a ritrasmettere dalla finestra di congestion avoidance.

Livello di rete - il piano dei dati:

Il livello di trasporto viene interpretato dagli end system, mentre il livello di rete viene interpretato dai router.

Le principali funzionalità del router sono:

- Esaminare i campi del pacchetto IP.
- Eseguire l'operazione di inoltro del pacchetto in modo tale che si abbia una comunicazione end to end, le funzionalità di forwarding e Routing sono due delle principali funzionalità offerte dal livello di rete.

A livello di rete c'è una divisione delle funzionalità tra il piano dati e il piano di controllo.

- **Piano dati:** offre tutte le operazioni di valenza locale per ogni singolo router, sono le operazioni che portano ad un inoltro corretto dei pacchetti.
- **Piano di controllo:** si occupa di tutte le operazioni network wide, non limitate al singolo nodo ma che riguardano la rete come sistema, per esempio l'instradamento che ha una valenza globale.

Il piano di controllo offre anche operazioni di scambio di messaggi di controllo attraverso due approcci:

1. **Approccio tradizionale:** prevede che il piano di controllo sia implementato in tutti i router.
2. **Approccio innovativo:** prende il nome di **software defined networking**, il piano di controllo dei dispositivi viene spostato dai dispositivi e viene centralizzato in un server che prende il nome di controllore. Si ha un dispositivo centralizzato che è in grado di comunicare con i vari router.

Software defined networking (SDN) per il piano di controllo.

Non dispone di un piano di controllo distribuito in ognuno dei singoli nodi, ma un unico piano di controllo centralizzato che è eseguito in un controllore remoto, il quale si interfaccia con ogni router per ricevere informazioni relativamente al loro stato.

Piano dei dati:

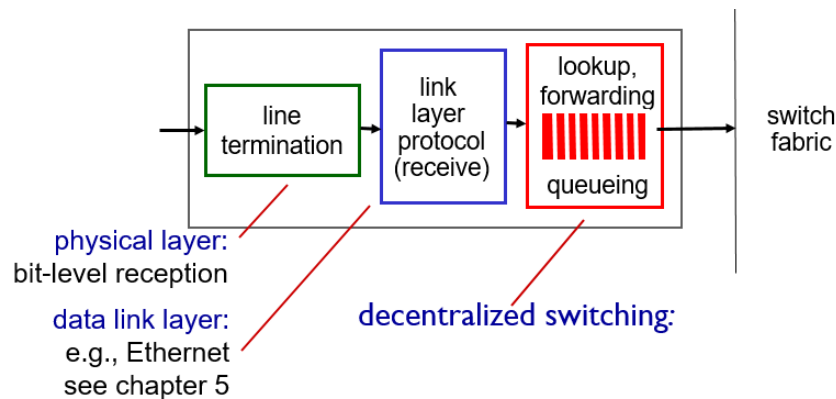
Con il modello di rete si possono implementare dei servizi molto sofisticati come una consegna garantita con annessa una soglia di ritardo fissato, questi servizi vengono offerti ai singoli datagrammi. Per quanto riguarda un insieme di datagrammi è garantita una consegna in ordine e una

banda minima per il flusso. A livello di rete ho un livello di servizio best effort, cerco di garantire i servizi, se non riesco pazienza.

Architettura del piano dati:

Avendo un certo numero di porte in input, e una serie di porte in output attraverso l'**high-speed switching fabric** è possibile interconnetterle.

Sul piano dati ho l'obiettivo di svolgere le operazioni il più velocemente possibile, per fare in modo che vengano eseguite in modo veloce è consigliabile utilizzare un'implementazione hardware, a differenza del piano di controllo dove si può lavorare in un time frame molto più rilassato nell'ordine dei millisecondi e pertanto c'è un'implementazione di tipo software.



All'interno dello switching fabric si implementano delle reti di commutazione che non sono full mesh (ogni nodo della rete è collegato direttamente a ogni altro nodo), ma hanno delle topologie specifiche che richiedono un numero inferiore di collegamenti e presentano una proprietà fondamentale, ovvero che i nodi della rete non siano bloccanti. Ciò permette di inviare un pacchetto da un'interfaccia a un'altra indipendentemente da quello che sta accadendo sulle altre reti.

La suddivisione tra porte input e output è una suddivisione logica tutte le porte possono fungere da input e output.

Porte di input:

Ciò che arriva sulla porta di input è un insieme di bit, c'è un box chiamato terminazione di linea che ha il compito di ricevere correttamente i bit e farli diventare intellegibili dai livelli più alti. Una volta che sono stati ricevuti vengono passati al livello di collegamento, successivamente il datagramma viene decapsulato e arriva nel box rosso.

Funzionalità di lookup: si usano i campi dell'header per capire verso quale porta dell'output è necessario inviare il datagramma attraverso un meccanismo chiamato **match plus action**. Il processing dei pacchetti deve avvenire a **line speed** ovvero è necessario limitare il più possibile l'accodamento dei pacchetti sull'interfaccia di input.

Tuttavia, non è sempre possibile perché i datagrammi certe volte arrivano più velocemente di quanto non vengano smaltiti. Si può ovviare a questo problema mantenendo le code di arrivo più scariche.

Quando vogliamo effettuare l'inoltro, con il protocollo IP si adotta un approccio di tipo **destination based forwarding**, che consiste nel prendere una decisione sull'interfaccia in output verso cui mandare il datagramma basandosi esclusivamente sull'indirizzo IP di destinazione del datagramma.

Accodamento nelle porte di input:

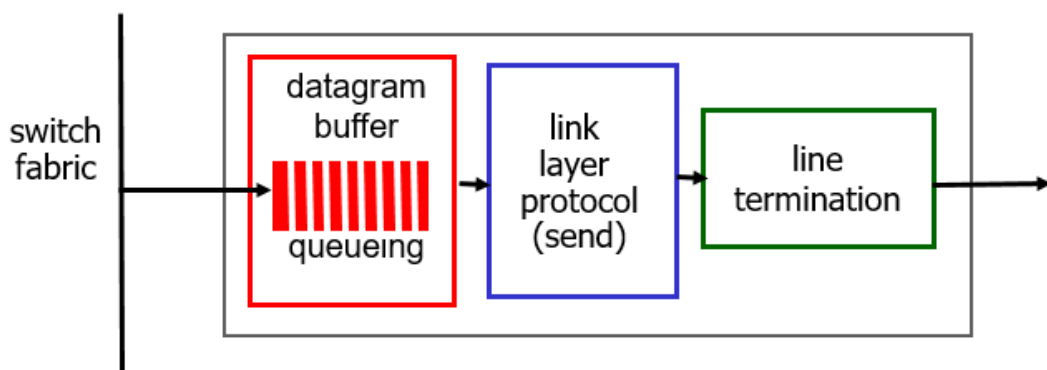
Per mantenere il livello delle code più basso possibile, si può dimensionare lo switching fabric.

Avendo un numero di porte di input pari a N e ognuna di questa riceve datagrammi con una banda pari a R , se lo switching fabric è in grado di effettuare le sue operazioni ad una velocità pari a NR , si verifica una condizione per la quale l'accodamento nelle porte di input è nullo. Nel caso questa condizione non sia garantita si può verificare un ritardo di accodamento oppure buffer overflow.

Tuttavia, è possibile riscontrare un altro problema: **problem of head of line blocking (HOL)** è un problema che porta ad aumentare il tempo di cui un datagramma, accodato in una porta di ingresso, necessita per essere inviato in una porta di output.

Accodamento nelle porte di output:

In caso di accodamento dei pacchetti nella coda si verificherà presto un buffer overflow; pertanto, vengono adottate delle politiche di **dropping**. Attraverso tali politiche si decide di scartare dei pacchetti prima di raggiungere il buffer overflow per quel traffico che è di più bassa priorità in modo tale da non scartare i pacchetti quando il buffer è pieno rischiando di eliminare pacchetti ad alta priorità.



Discipline di scheduling:

Attraverso le discipline di scheduling vengono gestiti i pacchetti a diversa priorità quando devono essere pescati dalla coda per poterli trasmettere. Esistono diverse discipline di scheduling che vanno a scegliere in modo intelligente quali pacchetti trasmettere sull'interfaccia, solitamente si adottano delle discipline che prevedono che la banda venga utilizzata maggiormente per i pacchetti ad alta priorità.

I protocolli di rete si dividono in un modello a clessidra; infatti, a livello applicativo e a livello di trasporto così come a livello di data link ci sono una grande quantità di protocolli mentre a livello di rete si verifica un "bottleneck" tanto da avere un unico protocollo disponibile, il protocollo IP.

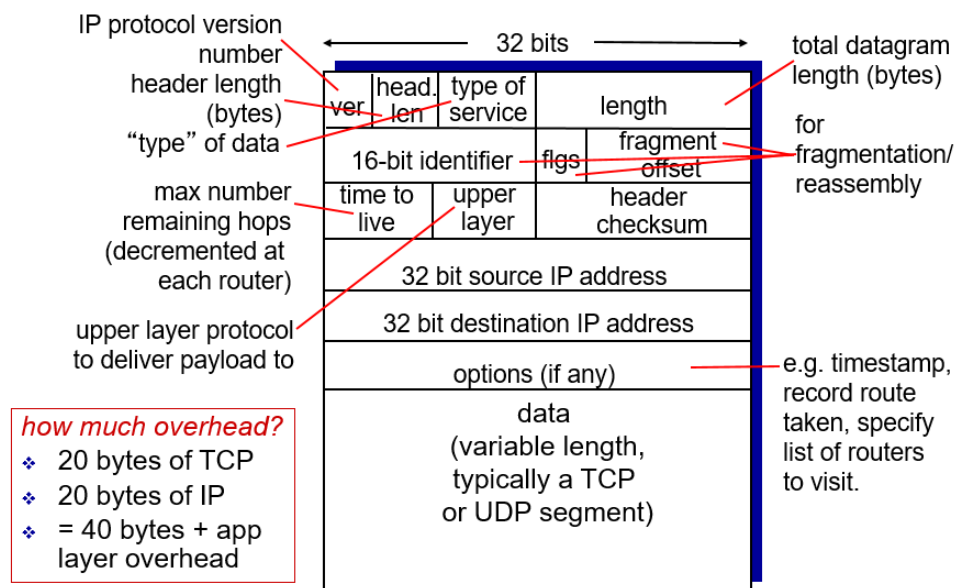
Header del datagramma IP:

Esistono due diverse versioni del protocollo IP, ipv4 e ipv6, al giorno d'oggi ipv4 è il protocollo più utilizzato nella rete internet.

L'header del datagramma IP è così composto:

- **Ver:** sono 4 bit che rappresentano la versione del protocollo.
- **Header length:** restituisce la lunghezza dell'Header, il campo header length ha delle opzioni che permettono di creare degli header di dimensione variabile.
- **Type of service:** sono 8 bit che servono per implementare la qualità di servizio in rete, vengono utilizzati per definire dei datagrammi con priorità differenti.
- **Time to live:** è un campo composto da 8 bit, tale campo può assumere un valore tra 0 e 255 e viene decrementato ogni volta che il pacchetto attraversa un router differente.

- **Upper layer:** sono 8 bit utilizzati per indicare il protocollo di livello superiore, il valore upper layer viene utilizzato per effettuare la multiplazione e la demultiplazione verso il corretto protocollo di trasporto.
- **Identifier:** è un campo di 16 bit che implementa la frammentazione, ogni router può frammentare un datagramma in datagrammi diversi. Al giorno d'oggi tale tecnica non è più utilizzata perchè si dimensiona il pacchetto a 1500 byte in quanto questa è la dimensione massima delle trame di livello ethernet.
- **header checksum:** necessario per il controllo degli errori, esegue il controllo solo sull'header quindi se sono presenti dei bit corrotti sul payload non me ne accorgo.
- **Source ip address** 32 bit.
- **Destination ip address** 32 bit.
- **Options:** in questo campo è possibile inserire la rotta dei router che sono stati attraversati.



Come è formato un indirizzo IP (ip addressing):

L'indirizzo ip è un identificatore di 32 bit che è associato a ogni interfaccia di un router o di un host. Un'interfaccia è una connessione tra un host o un router e un link fisico, i termini porta e interfaccia sono sinonimi.

Gli indirizzi ip sono rappresentati nel seguente modo:

Viene utilizzata una notazione decimale puntata.

Esempio:

233.1.1.1

Prendo i 32 bit, li divido in sottogruppi di 8 bit (1 byte) ciascuno e rappresento in decimale ognuno di questi byte aggiungendo il punto tra ogni valore.

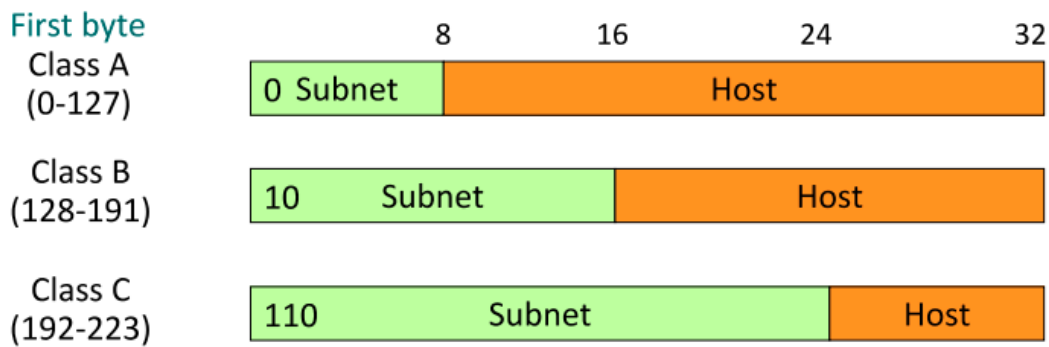
Come sono assegnati gli indirizzi IP nella rete?

Le interfacce sono connesse fisicamente, in particolare le interfacce che non hanno di mezzo un router, quest'ultime fanno tutte parte della stessa **subnet (sottorete)**.

Una parte dell'indirizzo IP riguarda la sottorete, l'altra riguarda l'host. La parte di sottorete sono i bit più significativi a sinistra, la parte di host i restanti bit a destra. \24 è una rappresentazione che determina quanti bit appartengono alla parte di sottorete e quanti alla parte di host.

IP addressing classful:

L'indirizzamento classful è un metodo di assegnazione degli indirizzi IP che si basa sulla divisione dello spazio di indirizzamento IPv4 in classi specifiche, identificate nel seguente modo:



IP addressing classless (CIDR):

Gli indirizzi ip vengono suddivisi con il metodo **CIDR (classless interdomain routing)**. La porzione di sottorete può avere un lunghezza arbitraria che viene rappresentata come \24.

Esempio:

200.23.16.0/23

I primi 23 bit rappresentano il prefisso (parte di sottorete) la restante parte è relativa all'host.

C'è un'altra rappresentazione che si chiama **subnet mask**:

Tale rappresentazione prevede di avere il valore 1 per i bit che rappresentano la sottorete e un valore 0 per la parte dell'host.

Indirizzi IP speciali:

- **Indirizzo di rete:** è un indirizzo in cui tutta la parte relativa all'host ha valore 0.
- **Indirizzo di broadcast:** la parte di host è posta tutta a valore 1.
- **Indirizzi di broadcast limitato:** il valore dei 32 bit è solo 1 (**255.255.255.255**), in questo caso il broadcast riguarda gli host all'interno della stessa sottorete.

Inoltro basato sulla destinazione (destination based forwarding):

Specifica come avviene l'inoltro all'interno dei router, in cui non si ha un singolo indirizzo ip ma un insieme di indirizzi ip. Per ogni range indirizzi ip si ha l'interfaccia di output del datagramma se si verifica un match. Un match si verifica quando i bit rappresentati in tabella sono gli stessi dell'indirizzo ip. Nel caso l'indirizzo ip non combaciasse con nessuno di quelli mostrati nella tabella si utilizza una **default route**.

Quando si verifica che un indirizzo ip combacia con due interfacce differenti si utilizza la regola del **longest prefix match** ovvero si prende il prefisso più lungo, quello con un numero maggiore di bit.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

Protocollo DHCP (Dynamic Host Configuration Protocol):

DHCP è un protocollo di rete utilizzato per assegnare dinamicamente indirizzi IP all'interno di una rete. Durante il processo di assegnazione di un indirizzo ip, DHCP prevede l'invio di quattro messaggi specifici tra il client DHCP (dispositivo che richiede un indirizzo) e il server DHCP (dispositivo che assegna gli indirizzi).

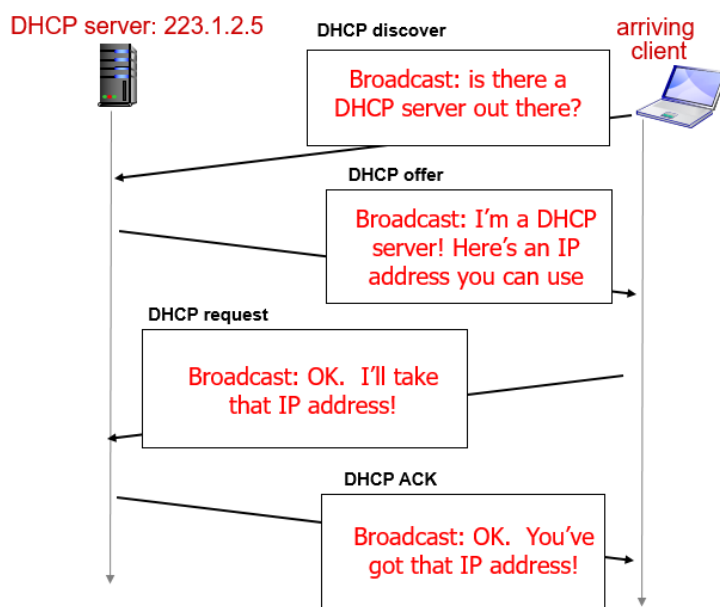
L'indirizzo ip che viene assegnato ha un **lease time**, ovvero ha un orizzonte temporale allo scadere del quale viene rimosso in modo tale che possa essere assegnato ad un altro utente.

DHCP richiede quattro messaggi:

- 1) **DHCP Discover:** Il client DHCP invia un messaggio di “DHCP Discover” in broadcast all'interno della rete locale. In questo messaggio, il client indica che sta cercando un server DHCP disponibile per ottenere un indirizzo IP.
- 2) **DHCP Offer:** Il server DHCP riceve il messaggio “DHCP Discover” e risponde con un messaggio “DHCP Offer”. Questo messaggio contiene l'indirizzo IP proposto per il client, insieme ad altre informazioni di configurazione, come la durata del lease (il periodo di tempo per cui l'indirizzo IP è assegnato al client).

Se un client ha già un indirizzo ip e vuole rinnovarne il lease time i primi due passaggi si possono saltare.

- 3) **DHCP Request:** Il client riceve uno o più messaggi “DHCP Offer” dai server DHCP disponibili e sceglie una delle offerte. Il client, quindi, invia un messaggio “DHCP Request” al server selezionato, confermando l'offerta e richiedendo ufficialmente l'assegnazione dell'indirizzo IP proposto.
- 4) **DHCP Acknowledge:** Il server riceve il messaggio “DHCP Request” e, se l'indirizzo IP proposto è ancora disponibile, risponde con un messaggio “DHCP Acknowledge”. Questo messaggio conferma ufficialmente l'assegnazione dell'indirizzo IP al client.



DHCP si può utilizzare anche per assegnare l'indirizzo ip al DNS server locale e per indicare il valore della subnet mask.

Come ottenere un indirizzo ip:

Un ISP assegna uno spazio di indirizzamento della sottorete, esegue un'operazione di **subnetting** in cui estende la parte di sottorete utilizzando dei bit che appartenerebbero alla parte di host.

Esempio:

Utilizzando uno /23 al posto di uno /20 posso indirizzare 2^3 reti. L'operazione può essere eseguita in modo ricorsivo.

ISP's block	<u>11001000 00010111 00010000</u>	00000000	200.23.16.0/20
Organization 0	<u>11001000 00010111 00010000</u>	00000000	200.23.16.0/23
Organization 1	<u>11001000 00010111 00010010</u>	00000000	200.23.18.0/23
Organization 2	<u>11001000 00010111 00010100</u>	00000000	200.23.20.0/23
...			
Organization 7	<u>11001000 00010111 00011110</u>	00000000	200.23.30.0/23

NAT (network address translation):

L'obiettivo del NAT è quello di fare in modo che tutti i dispositivi che appartengono ad una rete locale, condividano un unico indirizzo ipv4 ogni qual volta internet è coinvolto nella comunicazione. Con la definizione del NAT è sorta anche la distinzione tra indirizzi ip pubblici e indirizzi ip privati. Gli indirizzi ip pubblici sono quegli indirizzi ip che vengono assegnati alle interfacce direttamente connesse ad internet, gli indirizzi ip privati sono quegli indirizzi che appartengono alle reti locali.

Esistono tre diversi tipi di spazi di indirizzamento per indirizzi **ip privati**:

1. 10.0.0.0/8,
2. 172.16.0.0/12,
3. 192.168.0.0/16.

I vantaggi consistono in :

- Riduzione degli indirizzi ip necessari per interfacciarsi al mondo esterno.
- Si possono effettuare delle modifiche sulle reti locali senza impattare sul mondo esterno.
- Si può cambiare il l'ISP senza cambiare gli indirizzi ip nella rete privata.
- Tutti i dispositivi sotto NAT non possono essere raggiunti dal mondo esterno.

Un router che effettua l'operazione di natting esegue una traslazione per tutti i datagrammi che hanno un indirizzo ip di sorgente privato e partono da una rete locale. Quando un dispositivo nella LAN invia una richiesta a un server su Internet, il router NAT modifica l'indirizzo IP sorgente del pacchetto con il proprio indirizzo IP pubblico. Quando il server risponde alla richiesta, il router NAT controlla la tabella di traduzione NAT per determinare a quale dispositivo interno inoltrare la risposta.

Livello di rete - il piano di controllo

Algoritmi e protocolli di routing:

Gli algoritmi di routing permettono di determinare i cammini migliori tra la sorgente e la destinazione in una rete di router. **Gli algoritmi di routing** sono diversi dai **protocolli di routing**, quest'ultimi sono entità che permettono di determinare in che modo e come l'informazione deve essere condivisa tra i router per fare in modo che sia disponibile per gli algoritmi di routing e permetta loro di funzionare.

Costo dei collegamenti in un grafo:

Il costo di un link rappresenta il costo di un link diretto che collega due nodi e viene definito dall'operatore di rete che può decidere di adottare diverse politiche. Può scegliere i costi come inversamente proporzionali alla cogestione, andando a favorire quei link che sono meno congestionati, oppure direttamente proporzionali alla banda quindi cercando di agevolare collegamenti che hanno banda più elevata.

Classificazione degli algoritmi di routing:

Gli algoritmi di routing si possono classificare in due modi ortogonali tra di loro:

- **Algoritmi di tipo statico:** un algoritmo di routing è definito di tipo statico quando ci sono dei cammini che cambiano lentamente nel tempo, ciò avviene quando i costi sui link cambiano molto lentamente o addirittura non cambiano.
- **Algoritmi di tipo dinamico:** un algoritmo di routing è definito di tipo dinamico quando il costo viene aggiornato molto velocemente e gli stessi algoritmi devono rispondere a cambiamenti repentini nei costi dei link.

Esiste anche una seconda classificazione che divide **algoritmi di routing globali** e **algoritmi di routing decentralizzati**.

Algoritmi di routing decentralizzati: un algoritmo di routing decentralizzato è un algoritmo in cui si ha uno scambio esclusivamente con i nodi vicini e non con tutti i nodi della rete, questi algoritmi prendono il nome di **distance vector**, dove il vettore di distanza indica la distanza di ogni singolo nodo da tutti gli altri nodi, la distanza è il cammino di costo più basso.

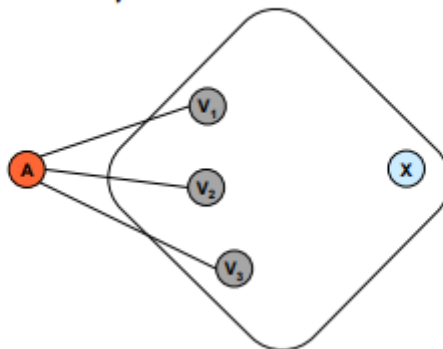
Algoritmi di routing globali: sono quegli algoritmi che portano ad avere una visione completa della topologia; quindi, ogni singolo router riesce a ricostruire la topologia e può cercare di capire quali sono i cammini migliori, questi algoritmi prendono il nome di link state.

Algoritmi distance vector:

Gli algoritmi distance vector si basano su una implementazione distribuita dell'algoritmo di **Bellman-Ford** che a sua volta si basa sull'equazione di Bellman-Ford.

Il nodo A è la sorgente mentre il nodo X è la destinazione, voglio creare un percorso di costo minimo, ovvero quel cammino che andando a sommare tutti i costi dei link dalla sorgente alla destinazione mi dia il costo minore tra A e X.

$$dist(A, X) = \min_V \{ dist(V, X) + c(A, V) \}$$



A è direttamente collegato ai nodi **v1**, **v2**, **v3** con dei link diretti che hanno un determinato costo, si sa che tra **v1 e X**, **v2 e X**, **v3 e X** esiste un cammino di costo minimo denominato **dist(V,X)**.

L'equazione di Bellman-Ford dice che la distanza tra A e X è uguale a tutte le distanze dei cammini che collegano i vari v ad X sommate al costo dei cammini che collegano A ai vari v.

Funzionamento:

Ogni nodo invia il suo vettore di distanza a tutti i suoi vicini (quelli con cui ha un collegamento diretto), quando un nodo ottiene un vettore di distanza da un suo vicino allora esegue nuovamente il calcolo dell'equazione di Bellman-Ford verso tutte le destinazioni in rete e valuta se ha avuto un cambiamento nel suo vettore di distanza. Se è avvenuto un cambiamento, questo vettore di distanza viene distribuito nuovamente a tutti i suoi vicini. Continuando con l'iterazione gli aggiornamenti ai

vettori di distanza calano e si verifica una convergenza tra il valore della distanza di un nodo X e la destinazione Y e il cammino di costo minimo tra il nodo X e il nodo Y. L'algoritmo dei vettori di distanza, si ferma autonomamente quando non si hanno più cambiamenti nei vettori di distanza.

Algoritmi link-state:

Il problema principale degli algoritmi distance vector viene risolto con altri algoritmi chiamati link state. Negli algoritmi distance vector viene calcolato il vettore di distanza verso tutti gli altri nodi della rete; tuttavia, viene comunicato esclusivamente ai nodi vicini. Nel caso di **algoritmi link state** comunico lo stato dei link (a quali nodi sono direttamente collegato e con un link di quale peso) inoltrando l'informazione in **selective flooding** a tutti gli altri nodi della rete.

Il vantaggio di questa implementazione risiede nel fatto che ogni singolo router può ricostruire la topologia completa della rete. Lo svantaggio risiede nel fatto che il numero di messaggi da inviare in rete risulta molto più elevato.

Successivamente si esegue l'algoritmo di **Dijkstra** che calcola l'albero dei cammini minimi da un nodo specifico, questo albero indica qual è il cammino più breve per poter raggiungere qualsiasi nodo.

Differenze tra algoritmi distance vector e algoritmi link state:

- Distance vector ha un'implementazione più semplice e intuitiva.
- Con distance vector i vettori di distanza vengono inviati solo ai vicini, quindi si hanno meno messaggi scambiati, nel caso di link state viene utilizzato selective flooding.
- Gli algoritmi link state sono generalmente più veloci a convergere rispetto agli algoritmi distance vector a causa della loro capacità di avere una visione globale della rete e di ricalcolare rapidamente i percorsi ottimali quando si verificano cambiamenti.

Protocolli di routing scalabili:

I router sono aggregati in regioni che prendono il nome di sistemi autonomi o domini. Per effettuare **l'instradamento all'interno di un sistema autonomo**, i router eseguono lo stesso protocollo intra dominio ad esempio (OSPF), i protocolli che si trovano in sistemi differenti usano protocolli intra-dominio differenti.

Per effettuare **l'instradamento tra sistemi autonomi differenti**, si utilizzano i **gateway router**, sono i router ai bordi di un sistema autonomo che permettono di collegare più sistemi autonomi tra di loro.

Problema di instradamento inter-AS:

Supponiamo di avere tre sistemi autonomi differenti, AS1, AS2, AS3 e un datagramma che va destinato all'esterno del sistema autonomo. Il datagramma deve essere inoltrato verso uno dei gateway router, nel caso ci fossero più gateway router mi avvalgo dei protocolli inter-AS per capire verso quale gateway router indirizzare il datagramma. I sistemi autonomi AS2 e AS3 comunicano con il sistema autonomo AS1 cercando di capire quali destinazioni possono essere raggiunte tramite il suo gateway router. Le informazioni giungono al gateway router del sistema autonomo 1 e vengono propagate ai router più interni in modo tale da poter indirizzare correttamente il datagramma.

Instradamento all'interno di un sistema autonomo:

I protocolli intra-AS prendono anche il nome di **interior gateway protocols**, i tre principali sono:

- **RIP (routing information protocol):** è un protocollo distance vector che non viene più utilizzato.
- **EIGRP:** è protocollo distance vector di CISCO.

- **OSPF (open shortest path first):** è un protocollo di routing di tipo link state.

OSPF:

OSPF è un protocollo di routing di tipo link state che utilizza il selective flooding per inoltrare i link state advertisement e l'algoritmo di Dijkstra per determinare il cammino di costo minimo in una rete.

Caratteristiche di OSPF:

- Tutti i messaggi di OSPF sono autenticati, ciò permette di prevenire la possibilità di immettere dei link state advertisement falsi che potrebbero portare ad alterare l'instradamento rispetto a quello corretto.

OSPF ha una gerarchia a due livelli:

- **Area locale:** insieme di router all'interno di un sistema autonomo.
- **Backbone:** interconnette le varie aree locali.

Viene creata questa gerarchia perché OSPF può essere utilizzato su sistemi autonomi di grandi dimensioni e in questo modo si riduce notevolmente il numero di messaggi inviati in flooding.

Funzionamento:

I router OSPF inoltrano in broadcast i Link State Advertisement che contengono informazioni sulla topologia della rete. Ogni router OSPF utilizza le informazioni ricevute nei link state advertisement per costruire una mappa completa della topologia della rete. Il router utilizza quindi l'algoritmo di Dijkstra per calcolare il percorso più breve verso tutte le destinazioni nella rete. Quando un router OSPF riceve un pacchetto di dati, utilizza la sua tabella di routing (che è stata calcolata dall'algoritmo di Dijkstra) per determinare il percorso migliore per il pacchetto e poi inoltra il pacchetto lungo quel percorso.

Protocollo di instradamento inter-AS: BGP (border gateway protocol):

BGP è l'unico protocollo di instradamento utilizzato inter-AS. Una sottorete che è collegata ad un sistema autonomo può rendere nota la sua esistenza ad altri sistemi autonomi tramite dei messaggi che vengono creati e propagati dal protocollo BGP.

BGP si scompone in due sotto protocolli:

1. **eBGP, exterior BGP:** per mezzo di questo protocollo si ottiene l'informazione sulla raggiungibilità di una sottorete dai sistemi autonomi vicini.
2. **iBGP, interior BGP:** si occupa di propagare l'informazione di raggiungibilità ricevuta da altri sistemi autonomi, e grazie a questa informazione è possibile determinare le rotte e i cammini verso le sottoreti seguendo determinate politiche.

I router di bordo sono interconnessi tra loro tramite il protocollo eBGP, mentre il protocollo iBGP viene utilizzato dai router all'interno del sistema per comunicare anche con i router di bordo.

Funzionamento BGP:

Quando si parla di BGP un concetto fondamentale è quello di sessione BGP in cui i router, denominati "**peers**" si scambiano messaggi di tipo BGP tramite una connessione TCP semipermanente.

BGP è un protocollo di tipo path vector che funziona similmente ai protocolli distance vectors ma su una scala più ampia. Per mezzo delle sessioni BGP vengono scambiati i percorsi verso le diverse reti di destinazione. Per effettuare l'instradamento inter-sistema autonomo, le decisioni non vengono prese in base alla performance bensì in base a scelte economiche e politiche.

Protocollo ICMP (internet control message protocol):

È un protocollo utilizzato per scambiare dei messaggi di controllo, oppure dei messaggi relativi alla rete e al suo stato, viene utilizzato principalmente per due scopi:

1. Per fare reporting degli errori.
2. Per fare diagnostica, per esempio il calcolo del ping. Il ping viene creato tramite l'invio di messaggi echo request che hanno una risposta in un messaggio echo reply, tenendo conto del tempo che intercorre tra l'invio e la ricezione del messaggio ho una stima del round trip time tra sorgente e destinazione.

Messaggi ICMP:

- **Dest port unreachable:** è un messaggio che viene generato quando effettuo un invio di messaggio a livello di trasporto UDP e non è stata aperta una socket alla destinazione.
- **TTL expired:** è un messaggio di errore che viene generato dai router o dagli host nel momento in cui andando a decrementare il valore del time to live all'interno del pacchetto IP si raggiunge il valore 0 e il pacchetto viene scartato.

Traceroute ICMP:

Sfrutta il messaggio TTL expired, per tracciare il percorso di un pacchetto tra sorgente e destinazione.

Funzionamento:

TTL è un campo nei pacchetti IP che rappresenta il numero massimo di router che un pacchetto può attraversare prima di essere scartato.

Quando traceroute invia un pacchetto con un TTL di 1, il primo router lungo il percorso lo scarta, decrementa il TTL a zero e invia un messaggio ICMP **“Time Exceeded”** al mittente. Questo messaggio include l'**indirizzo IP** del router che ha scartato il pacchetto.

Traceroute visualizza l'indirizzo IP del router e il tempo impiegato per raggiungerlo; quindi, incrementa il TTL e invia un nuovo pacchetto ICMP, ripetendo il processo fino a raggiungere la destinazione finale.

Quando il pacchetto raggiunge la destinazione, la destinazione invia una risposta ICMP **“Port Unreachable”** poiché il pacchetto ha tentato di raggiungere una porta non aperta.

Livello di collegamento e reti locali:

Il collegamento è un canale di comunicazione che interconnette nodi adiacenti su un cammino, in generale un collegamento può essere di tre tipi diversi:

- Wired
- Wireless
- LANs

Su diversi collegamenti, in un percorso che viene seguito da un datagramma, tra una sorgente e una destinazione possono essere usati differenti protocolli di collegamento. I protocolli differiscono a seconda del tipo di collegamento adottato, ad esempio i protocolli usati su un collegamento wireless sono profondamente diversi da quelli utilizzati su u collegamento in fibra ottica.

Servizi offerti a livello di collegamento:

- **Framing:** il datagramma deve essere incapsulato in una trama, viene aggiunta un'intestazione di livello 2 e in alcuni casi si può aggiungere anche un trailer.

- **Consegna affidabile tra nodi adiacenti:** su canali con un tasso di errore molto basso, non sono utilizzati protocolli che implementano la consegna affidabile, mentre su canali wireless con tassi di errori più elevati può essere implementato un meccanismo di consegna affidabile con ritrasmissione solo sul singolo link.
- **Controllo di flusso.**
- **controllo degli errori.**
- **Correzione di errore:** esistono dei meccanismi che permettono di correggere almeno in parte l'errore ristabilendo la corretta sequenza di bit.
- **Trasmissione half-duplex/full-duplex:** alcuni protocolli permettono la trasmissione full-duplex, mentre altri solamente half-duplex, ovvero in una sola direzione.

Protocolli ad accesso multiplo:

Esistono due diverse tipologie di collegamento fisico:

1. **Collegamento punto a punto:** collega due interfacce tra di loro
2. **Collegamento di tipo broadcast:** il collegamento è condiviso tra più host, un caso tipico è il canale radio.

Il link broadcast può essere anche fisico, in particolare un cavo coassiale su cui sono innestati diversi host, se uno trasmette tutti ricevono.

Come avviene la comunicazione su un canale di tipo broadcast:

Ho un canale condiviso tra più host, ciò implica che si potrebbe verificare la situazione di una trasmissione multipla che andrebbe a generare un'interferenza che prende il nome di **collisione**, il messaggio si sovrappone e non è più intellegibile per nessuno dei due host.

Il problema può essere risolto attraverso i **protocolli ad accesso multiplo** che vanno a governare l'accesso a un canale broadcast tra host di tipo diverso, in particolare i protocolli di accesso multiplo determinano qual è il nodo che può trasmettere.

Se vengono utilizzati protocolli ad accesso multiplo che richiedono l'invio di un messaggio per autorizzare la trasmissione di un nodo, questi messaggi usano il medesimo canale e quindi sono soggetti a collisioni.

Ci sono meccanismi più o meno semplici per governare le comunicazioni di più utenti, idealmente un protocollo ad accesso multiplo con una capacità di R bit al secondo dovrebbe comportarsi nel seguente modo:

- Se si ha un solo host che vuole trasmettere questo può trasmettere alla velocità massima.
- Se si hanno più host che vogliono trasmettere la banda deve essere distribuita in modo equo R/M .
- Il sistema deve poter essere decentralizzato perchè non ho necessità di avere un nodo che coordina le trasmissioni. Se questo nodo dovesse cadere il sistema continuerebbe a funzionare.

Esistono tre tipi diversi di protocolli ad accesso multiplo:

1. **Channel partitioning:** il canale viene suddiviso sulla base di una specifica risorsa, per esempio il tempo, la frequenza o il codice. Un host può trasmettere solo in determinati istanti di tempo non sovrapposti a quelli in cui possono trasmettere altri host.
2. **Random access:** sono i protocolli in cui si accetta di avere delle collisioni, il canale non è suddiviso tra i diversi utenti e bisogna gestire il caso in cui si verificano delle collisioni.

3. **Taking turns:** sono quei protocolli in cui a rotazione i diversi nodi acquisiscono la possibilità di andare a trasmettere e tendenzialmente sono dei protocolli in cui chi ha più dati da trasmettere può trasmettere più a lungo.

Protocolli ad accesso casuale:

Non richiedono un coordinamento tra i nodi, un host può trasmettere alla velocità R , velocità massima del canale in caso sia l'unico trasmittente. Nel caso in cui ci fossero due nodi che trasmettono in contemporanea si verifica una collisione. Tra i protocolli ad accesso casuale ci sono

- ALOHA - slotted aloha.
- CSMA - collision detection/collision avoidance.

Slotted ALOHA (versione migliorata di ALOHA):

Nel caso di Slotted-Aloha tutte le trame che devono essere inviate sono della stessa dimensione, il tempo è suddiviso in slot temporali di uguale dimensione e tale dimensione è la dimensione necessaria per trasmettere una trama.

È possibile trasmettere solo all'inizio di uno slot, se due nodi trasmettono nello stesso slot si ha una collisione. I nodi sono sincronizzati, ciò significa che devono condividere lo stesso orologio, il tempo è lo stesso per ognuno dei nodi.

Slotted-Aloha è un protocollo ad accesso casuale perchè quando un nodo ha una trama da trasmettere la trasmette nel primo slot disponibile, se si verifica una collisione il nodo ritrasmette la trama in uno slot successivo con una determinata probabilità p finché la ritrasmissione non va a buon fine.

Svantaggi:

- Ci sono collisioni.
- Alcuni slot vengono sprecati perchè non c'è ritrasmissione anche a condizioni favorevoli.
- Altri slot vengono sprecati perchè non sono utilizzati.
- Si ha una sincronizzazione tra i vari nodi.

Vantaggi:

- Se ho un solo nodo che deve trasmettere questo può trasmettere alla massima velocità del canale finché non si verificano collisioni,
- Il protocollo è fortemente decentralizzato perchè l'unica informazione che deve essere trasmessa è la sincronizzazione.

L'efficienza: è il rapporto tra gli slot in cui ho avuto un successo nella trasmissione rispetto al numero degli slot totali. L'efficienza massima del protocollo Slotted-Aloha è il 37%, il canale è agibile per la trasmissione il 37% del tempo.

Pure aloha:

Non richiede uno slotting dei tempi, si può trasmettere immediatamente, quando una trama arriva viene trasmessa immediatamente. Il problema consiste nell'aumento della probabilità di avere una collisione. L'efficienza è pari al 18%.

CSMA (carrier sense multiple access):

Il protocollo CSMA esiste in tre versioni differenti:

- **CSMA:** prevede un ascolto del canale prima della trasmissione, se il canale risulta libero si trasmette tutta la trama, se il canale è occupato si posticipa la trasmissione.

- **CSMA/CD:** aggiunge la funzione di collision detection che prevede l'ascolto del canale sia prima che durante la trasmissione, in questo modo ci si può rendere conto se si sta ricevendo una trasmissione da parte di qualcun altro.
- **CSMA/CA:** CSMA con collision avoidance.

CSMA/CD:

Nel momento in cui un nodo si rende conto che si verifica una collisione tra i segnali, si ha un tempo necessario per abortire la trasmissione, viene inviato un **segnale di jam** con un pattern definito che va ad avvisare tutti gli host che sta sperimentando una collisione e smette di trasmettere. In questo modo viene ridotto di molto il tempo sprecato con le collisioni.

Algoritmo CSMA/CD (ethernet):

Viene creata una trama dal livello di collegamento, viene ascoltato il canale e nel caso fosse libero può avvenire la trasmissione, altrimenti si aspetta fino a che il canale non viene liberato.

Nel momento in cui il canale risulta libero è necessario aspettare un lasso di tempo chiamato **inter-frame space (IFS)** necessario per fare in modo che due diverse trame siano distinguibili tra di loro. Se la trasmissione avviene senza alcuna collisione l'invio è completato, altrimenti ci si ferma e si invia il segnale di jam. Ogni singolo nodo entra nella fase di **binary exponential backoff** in cui si ritarda la trasmissione per un tempo che è tanto più grande tante sono le collisioni sperimentate dal nodo. Dopo la **m-esima** collisione viene scelto un numero **k** tra **(0, 1, 2, ..., 2^{m-1})** e si attende un tempo pari a **512k** bits prima di ritornare ad ascoltare il canale. All'aumentare del numero di collisioni aumenta esponenzialmente il tempo di attesa per la ritrasmissione.

Efficienza di CSMD/CD:

L'efficienza è massima, cioè tende a 1 tanto più il tempo di propagazione tra due nodi tende a 0 e il tempo per trasmettere una trama di massima dimensione tende ad infinito.

$$efficiency = \frac{1}{1 + 5 \frac{t_{prop}}{t_{trans}}}$$

Protocolli di accesso taking turns:

Un protocollo taking turns viene adottato ad esempio da bluetooth per collegare i vari dispositivi, la caratteristica di questi protocolli è l'assenza di collisioni. Esistono due tipologie di protocolli taking turns:

- **Protocolli polling:** Nei protocolli di tipo polling c'è una gerarchia composta da un nodo master (più intelligente) e dei nodi slave, se questi ultimi vogliono comunicare con il nodo master, il nodo master a turno invia un messaggio di polling ad ogni nodo slave. Gli svantaggi principali sono la presenza di overhead dovuto al messaggio di controllo inviato ai nodi slave per autorizzarli alla trasmissione, la latenza; infatti, è necessario aspettare che il master abbia interrogato tutti gli altri nodi e il single point of failure, se il master che governa la comunicazione dovesse fallire non funziona più il sistema.
- **Protocolli token passing:** un esempio protocollo token pass è il protocollo token ring, utilizzato per implementare le reti locali. Nei protocolli token pass tutti i dispositivi sono gerarchicamente dello stesso tipo e sono collegati tra di loro per mezzo di un anello logico: Il messaggio di controllo chiamato token viene passato seguendo il loop, in senso antiorario, a tutti gli altri nodi; quindi, ogni nodo è in grado di accedere ai dati. Gli svantaggi sono i medesimi dei protocolli polling, ovvero il token overhead, la latenza e il single point of failure rappresentato dal token.

Reti locali commutate:

Le reti locali vengono implementate tramite dei dispositivi attivi di livello due che prendono il nome di switch ed effettuano un'operazione di inoltramento delle trame da una destinazione ad un'altra.

MAC address:

La funzionalità degli indirizzi MAC è fare in modo che un datagramma sia in grado di andare da un'interfaccia ad un'altra fisicamente adiacente, ovvero raggiungibile senza passare da un router.

Gli indirizzi MAC sono composti da 48 bit e sono fisicamente associati all'interfaccia di rete, i 48 bit sono raggruppati in coppie di cifre esadecimali, ogni coppia rappresenta 4 bit.

L'indirizzo MAC è un indirizzo "fisico" perché è legato all'hardware di un dispositivo e non cambia, mentre l'indirizzo IP è un indirizzo "logico" perché può cambiare a seconda della rete a cui il dispositivo è connesso.

Local area network (LAN):

Gli indirizzi MAC vengono assegnati dalla IEEE, questo istituto alloca spazi di indirizzamento MAC a dei vendor che vogliono produrre delle schede di rete.

L'indirizzamento piatto per gli indirizzi MAC garantisce la portabilità, spostandosi attraverso reti locali ho un cambiamento solo nell'indirizzo IP e non nell'indirizzo MAC.

Address resolution protocol (ARP):

All'interno di una rete locale la comunicazione avviene a livello due ed è necessario utilizzare l'indirizzo MAC per comunicare. Per recuperare l'indirizzo MAC associato all'host conoscendo l'indirizzo IP si utilizza il protocollo ARP. Ogni host e router nella rete ha una ARP table con un'associazione tra l'indirizzo IP di un'interfaccia e l'indirizzo MAC.

Funzionamento protocollo ARP:

Suppongo tutte le tabelle ARP vuote, A vuole inviare un datagramma a B, viene utilizzato il protocollo ARP per trovare l'indirizzo MAC associato all'indirizzo IP di B.

A invia un messaggio di **ARP query**, in cui viene incluso l'indirizzo IP di B. Il messaggio è incapsulato in una trama, come destinazione della trama si utilizza l'indirizzo broadcast, tutti i nodi che si trovano sulla LAN ricevono la trama con indirizzo unicast. B risponde ad A con un messaggio di **ARP response** (viene inviato specificatamente ad A) all'interno di questo messaggio inserisce il MAC address.

Ethernet:

Si tratta del protocollo di livello 2 che viene utilizzato maggiormente nelle reti locali, supporta una velocità di trasmissione fino a 400 Gbps e con un singolo chip ethernet è possibile supportare diverse velocità di trasmissione.

Ethernet frame structure:

- **Preambolo:** 8 byte di informazione che includono un pattern identico, i primi 7 byte sono un'alternanza di 10, mentre l'ultimo byte ha come ultimi due bit 11. L'utilità del preambolo è duplice:
 - Se ascoltando il canale riconosco il pattern, capisco che ho una trama che sta per arrivare.
 - Fare in modo che ci sia una sincronizzazione tra ricevitore e trasmettitore. Ethernet supporta velocità di tipo diverso, pertanto, utilizzo i byte del preambolo per rendere compatibile la velocità di invio e di ricezione degli host.
- **Indirizzo MAC di destinazione/sorgente:** 12 byte, 6 per l'indirizzo sorgente e 6 per l'indirizzo destinazione

- **Campo type:** è un campo di 2 byte utilizzato per la multiplazione/demultiplazione, ovvero per capire a quale protocollo di livello superiore deve essere inviata la trama una volta decapsulata.
- **CRC:** sono 4 byte che includono codici di ridondanza ciclica per la detection degli errori.
- **Payload:** datagramma ricevuto dal livello IP. Il payload in ethernet va da 46 a 1500 byte, 1500 è l'unità di trasmissione massima, se arrivano datagrammi con dimensione maggiore vengono frammentati. Se i dati che devono essere incapsulati sono inferiori ai 46 byte, si inseriscono dei **padding bytes** (byte di riempimento) che hanno pattern regolari per raggiungere la dimensione minima di 46 byte.

La dimensione minima di un frame ethernet è di **72 byte** compreso il preambolo, mentre l'header di un frame ethernet è di **26 byte** compreso il preambolo.

Ethernet switch:

Uno switch è un dispositivo attivo di livello di collegamento, adotta una modalità store and forward, prima di inviare una trama deve averla ricevuta interamente. Per recapitare la trama, legge l'indirizzo MAC e inoltra la trama su una o più interfacce in uscita.

La presenza degli switch è completamente sconosciuta agli host e ai router, ciò significa che gli switch non hanno associato alcun indirizzo MAC alle loro interfacce. Gli switch non appena vengono collegati alla rete si auto configurano.

Supponiamo che io voglia avere una connessione simultanea tra A-A' e B-B' lo switch garantisce che tale connessione possa avvenire senza alcuna collisione.

Switch forwarding table:

Lo switch ha una tabella di inoltri chiamata switching table, questa contiene l'indirizzo di MAC e l'interfaccia, la tabella viene popolata tramite un meccanismo chiamato self-learning e forwarding.

La prima fase è quella di **learn:** attraverso le trame che passano nel router, questo apprende quali host possono essere raggiunti e per mezzo di quali interfacce.

La seconda fase è quella di **forwarding:** registro il link di ingresso e il source MAC address dell'host che sta inviando la trama e lo metto nella tabella di inoltri. Se nella tabella di inoltri ho un'associazione allora posso inviare sull'interfaccia se questa è diversa da quella da cui ho ricevuto la trama, se invece l'interfaccia di invio è la stessa da cui ho ricevuto la trama, quest'ultima viene scartata. Se invece non ho alcuna associazione invio la trama verso tutte le interfacce eccetto quella da cui l'ho ricevuta effettuando un'operazione di **flooding**.

Confronto switches routers:

I router sono dei dispositivi di livello tre e vanno ad analizzare le intestazioni a livello 3 mentre gli switch a livello 2. Entrambi utilizzano le forwarding tables, per i router sono popolate attraverso gli algoritmi di routing mentre per gli switch tramite il meccanismo di learning e forwarding.

Sistemi Operativi: strutture e servizi

Sistema operativo:

È un insieme di programmi che gestisce gli elementi fisici del computer. Fornisce una piattaforma di sviluppo per i programmi applicativi che permette loro di condividere ed astrarre le risorse hardware, inoltre agisce da intermediario tra utenti e computer.

Struttura del sistema operativo:

Un sistema operativo comprende almeno:

- **Kernel:** è il nucleo del sistema operativo che gestisce l'hardware ed offre ai programmi i servizi per poterlo usare in maniera condivisa e astratta.
- **Middleware:** è un insieme di servizi di alto livello che semplificano la programmazione di applicazioni.
- **Programmi di sistema:** non sempre in esecuzione, offrono ulteriori funzionalità di supporto e di interazione utente con il sistema.

Servizi del sistema operativo:

- **Controllo processi:** questi servizi permettono di caricare in memoria un programma, eseguirlo, identificare la sua terminazione e registrarne la condizione di terminazione.
- **Gestione file:** questi servizi permettono di leggere, scrivere, e manipolare files e directory.
- **Gestione dispositivi:** questi servizi permettono ai programmi di effettuare operazioni di input/output, ad esempio leggere da/scrivere su un terminale.
- **Comunicazione tra processi:** i programmi in esecuzione possono collaborare tra di loro scambiandosi informazioni: questi servizi permettono ai programmi in esecuzione di comunicare.
- **Protezione e sicurezza:** permette ai proprietari delle informazioni in un sistema multiutente o in rete di controllarne l'uso da parte di altri utenti e di difendere il sistema dagli accessi illegali.
- **Allocazione delle risorse:** alloca le risorse hardware.
- **Rilevamento errori:** gli errori possono avvenire nell'hardware o nel software, quando avvengono il sistema operativo deve intraprendere opportune azioni.
- **Logging:** mantiene traccia di quali programmi usano quali risorse, allo scopo di contabilizzarle.

Chiamate di sistema e API:

Le chiamate di sistema sono delle funzioni invocabili in un determinato linguaggio di programmazione attraverso le quali il kernel offre i propri servizi. I programmi però non utilizzano direttamente le chiamate di sistema ma delle librerie di middleware dette (**Application Program Interface – API**). (Win32 su Windows, POSIX, su Unix, Linux, MacOS)

Differenza tra chiamate di sistema e API:

- Le API sono esposte dal middleware, le chiamate di sistema dal kernel.
- Le API usano le chiamate di sistema nella loro implementazione.
- Le API sono standardizzate.
- Le API sono stabili al controllo delle chiamate di sistema che variano in base al sistema operativo.
- Le API offrono funzionalità ad alto livello e più semplici da usare, le chiamate di sistema offrono funzionalità più elementari e più complesse.

Programmi di sistema:

Gli utenti solitamente utilizzano il sistema operativo attraverso i programmi di sistema, questi permettono infatti di avere un ambiente più conveniente per l'esecuzione, lo sviluppo e la gestione delle risorse.

Tipologie di programmi di sistema:

- **Interfaccia utente (UI):** permette agli utenti di interagire con il sistema stesso, può essere di tipo grafico (GUI) o a riga di comando (CLI).
- **Gestione dei file:** editor di testo, programmi per la manipolazione del contenuto dei file.

- **Caricamento ed esecuzione dei programmi:** loader assoluti rilocabili, linker e debugger.
- **Ambienti di supporto alla programmazione:** compilatori, assembleri, debugger.
- **Comunicazioni:** forniscono i meccanismi per creare connessioni tra utenti, programmi e sistemi.
- **Servizi in background:** fornisco servizi quali verifica dello stato dei dischi, scheduling di jobs, logging.

Interfaccia utente:

L'interprete dei comandi (**Shell**) permette agli utenti di impartire in maniera testuale delle istruzioni al sistema operativo, in molti sistemi operativi è possibile scegliere quale interprete dei comandi utilizzare. Esistono due modi per implementare un comando:

- **Built-in:** l'interprete esegue direttamente il comando (tipico di Windows).
- **Programma di sistema:** l'interprete manda in esecuzione il programma (tipico delle shell Unix e Unix-like).

Processi e thread – i servizi:

Processo:

Un processo è un'entità attiva astratta definita dal sistema operativo allo scopo di eseguire un programma.

Differenza tra programmi e processi:

- **Il programma è un'entità passiva**, ovvero un insieme di istruzioni contenute in un file sorgente o eseguibile. **Un processo è un'entità attiva**, un esecutore di programma o un programma in esecuzione.
- Uno stesso programma può dare origine a processi diversi.

Struttura di un processo:

Un processo è costituito da diverse parti:

- Stato dei registri del processore che esegue il programma, incluso il program counter.
- Stato dell'immagine del processo, ovvero la regione di memoria utilizzata dal programma.
- Risorse del sistema operativo in uso al programma.
- Diverse informazioni sullo stato del processo per il sistema operativo.

Processi distinti hanno immagini distinte, operano in zone di memoria centrale separate. Le risorse del sistema operativo invece possono essere condivise tra processi.

Immagine di un processo:

- **Text section:** contenente il codice macchina del programma.
- **Data section:** contenete le variabili globali.
- **Heap:** contenente la memoria allocata dinamicamente durante l'esecuzione.
- **Stack:** contenente parametri, variabili locali e indirizzo di ritorno delle varie procedure che vengono invocate durante l'esecuzione del programma.

Operazioni sui processi:

I sistemi operativi di solito forniscono delle chiamate di sistema con le quali un processo può creare, terminare e manipolare altri processi. Dal momento che solo un processo può creare un altro processo, all'avvio il sistema operativo crea dei processi **“primordiali”** dai quali tutti i processi utente e di sistema vengono progressivamente creati.

Creazione di processi:

I processi sono organizzati in maniera gerarchica, un processo padre può creare altri processi figli formando un albero di processi.

Terminazione di processi:

I processi chiedono la loro terminazione al sistema operativo, un processo padre può attendere o meno la terminazione di un processo figlio, oppure può forzarla. Alcuni sistemi operativi non consentono ai processi figli di esistere dopo la terminazione di un padre.

- Se un processo termina, ma il padre non lo sta aspettando, tale processo è detto **zombie**.
- Se un processo termina prima di suo figlio, il figlio è **orfano**.

Comunicazione interprocesso:

I processi possono essere indipendenti oppure possono cooperare.

- Un processo coopera se il suo comportamento influenza o è influenzato da altri processi.
- Per cooperare il sistema deve mettere a disposizione primitive di comunicazione interprocesso (IPC).

IPC tramite memoria condivisa:

Viene stabilita una zona di memoria condivisa tra i processi che intendono comunicare, la comunicazione è controllata dai processi e non dal sistema operativo.

IPC tramite message passing:

Il message passing permette ai processi sia di comunicare che di sincronizzarsi attraverso la mediazione del sistema operativo. Per comunicare tra di loro due processi necessitano di un link di comunicazione e utilizzano le operazioni di send e receive messe a disposizione dall' OS.

PIPE:

Le pipe sono canali di comunicazione tra processi, sono una forma di message passing possono presentarsi in due diverse tipologie:

1. PIPE convenzionali:

Sono delle pipe unidirezionali non accessibili al di fuori del processo creatore e sono di solito condivise con un processo figlio attraverso una fork.

2. Named PIPE:

Sono delle pipe bidirezionali che esistono dopo la terminazione del processo che le ha create e non richiedono una relazione padre/figlio tra i processi che le usano. In Unix sono Half-duplex (un solo processo scrive mentre gli altri leggono) in Windows sono full duplex.

Multithreading:

Supponendo che un processo possa avere molti processori virtuali, più istruzioni possono essere eseguite insieme e quindi il processo può avere più percorsi (THREAD) di esecuzioni concorrenti.

- I thread di uno stesso processo condividono la memoria globale, la memoria contenente il codice e le risorse ottenute dal sistema operativo.
- Ogni thread ha il proprio stack, altrimenti le chiamate a subroutine di un thread interferirebbero con quelle di un altro thread concorrente.

Librerie di thread:

sono le API fornite al programmatore per creare e gestire thread:

- Posix: pthreads.

- Windows: threads.

API per il multithreading:

- **Fork:** duplica l'intero processo, compresi tutti i thread che sono presenti nel processo chiamante.
- **Execute:** invocata da un thread, termina tutti i thread del processo precedentemente in esecuzione.

Cancellazione dei thread:

L'operazione di cancellazione di un thread determina la terminazione prematura del thread, può essere invocata da un altro thread e può essere di due tipi:

- **Cancellazione asincrona:** il thread che riceve la cancellazione viene terminato immediatamente.
- **Cancellazione differita:** un thread che supporta la cancellazione differita deve controllare periodicamente se esiste una richiesta di cancellazione pendente e in tal caso terminare la propria esecuzione.

Gestione della memoria – i servizi:

Perché un programma possa andare in esecuzione esso deve avere a disposizione:

- Il processore, per eseguire il codice.
- La memoria centrale, per memorizzare il codice e i dati sul quale il codice opera.

Nei moderni sistemi operativi molti programmi sono contemporaneamente in memoria in uno stesso istante. Il sistema operativo deve, pertanto, allocare porzioni di memoria centrale ai diversi processi in funzione delle necessità di tali processi.

Binding degli indirizzi:

In presenza di molti programmi in memoria, il sistema operativo di regola carica uno stesso programma, in momenti diversi, in diverse aree di memoria. Per far sì che un'istruzione macchina di un programma faccia riferimento a una certa locazione di memoria anche se il suo indirizzo non è noto a priori ci sono due possibilità:

- 1) Il compilatore produce codice indipendentemente dalla posizione (**PIC**) **position independent code**, ossia codice macchina che usa solo indirizzi di memoria relativi e che quindi funziona correttamente in qualsiasi locazione di memoria venga caricato.
- 2) Produrre codice dipendente dalla posizione e tradurre gli indirizzi dipendenti dalla posizione negli indirizzi corretti. Questa operazione è detta **binding degli indirizzi**.

L'associazione degli indirizzi può essere fatta in tre momenti diversi:

- **In compilazione:** il linker, a partire dall'indirizzo di caricamento, effettua il binding e genera codice assoluto.
 - Si tratta di una soluzione semplice, ma se cambia l'indirizzo di caricamento il codice va ricompilato, si possono, ad esempio, avere n versioni per n diversi indirizzi di caricamento.
- **In caricamento:** il loader utilizzando delle tabelle che il compilatore lascia all'interno dell'eseguibile, esegue un'operazione di riscrittura.
 - Permette di variare liberamente l'indirizzo di caricamento da esecuzione ad esecuzione, ma è una soluzione lenta che non permette di relocare l'immagine di un

processo durante la sua esecuzione; inoltre, l'eseguibile deve contenere delle opportune tabelle che indichino le istruzioni macchina da modificare.

- **In esecuzione:** il binding viene effettuato dall'hardware dinamicamente mentre il codice viene eseguito, richiede un hardware speciale chiamato memory management unit.
 - Soluzione rapida che permette di rilocare l'immagine di un processo anche durante la sua esecuzione e di proteggere la memoria centrale non assegnata ad un processo, ma richiede il supporto dell'hardware.

Il binding in esecuzione è quello di fatto usato in tutti i sistemi operativi moderni.

Spazio di indirizzamento virtuale:

Nei sistemi operativi moderni ogni processo ha uno spazio di indirizzamento virtuale, **virtual address space (VAS)**, indipendente dagli indirizzi fisici della memoria centrale nella quale l'immagine è caricata. Tale spazio di indirizzamento si estende dall'indirizzo 0 al massimo indirizzo consentito dall'architettura del processore. Lo spazio di indirizzamento virtuale di un processo è di regola molto più ampio della memoria centrale, ciò comporta che buona parte dello spazio di indirizzamento virtuale non è utilizzabile (parte compresa tra lo stack e lo heap) dal processo perché non è associato a nessuna regione di memoria centrale.

Memory mapping:

I sistemi operativi mettono a disposizione API per mappare una regione inutilizzabile del VAS su memoria centrale, così che diventi utilizzabile, esistono anche delle API come **mmap** che permettono di mappare una regione del VAS sul contenuto di un file.

L'API mmap permette di mappare una regione ancora non utilizzata del VAS su:

- Memoria centrale.
- Un file.
- Memoria condivisa.

File system - i servizi:

Il file system è il metodo attraverso il quale il sistema operativo memorizza in linea i dati e i programmi, è costituito da:

- Un insieme di file.
- Una struttura delle directory che organizza i file.

File:

Un file è una unità di memorizzazione logica, un insieme di informazioni correlate, registrate in memoria secondaria, alle quali è stato dato un nome, è a sua volta costituito da una sequenza record, righe, bit o byte, il cui significato è definito dal creatore del file.

Attributi dei file:

- Nome.
- Identificatore.
- Tipo.
- Locazione.
- Dimensione.
- Protezione.
- Ora, data, utente.
- Attributi estesi.

Le informazioni dei file sono memorizzate nelle directory.

Lock dei file:

È possibile nel caso in cui un file venga aperto da due processi contemporaneamente che i due processi vadano in corsa critica, se un certo processo legge e un altro scrive sullo stesso file è necessaria una sincronizzazione; pertanto, i sistemi operativi mettono a disposizione dei file dei lock particolari che permettono di bloccare delle regioni di file o il file completo.

Tipi di lock:

- **Lock condiviso:** detto anche lock di lettura; più processi possono acquisirlo, proibisce l'acquisizione di un lock esclusivo.
- **Lock esclusivo:** detto anche lock di scrittura; solo un processo alla volta può acquisirlo, proibisce l'acquisizione di un lock condiviso.
- **Lock obbligatori (mandatory):** il sistema operativo impedisce l'accesso al file ai processi che non detengono il lock.
- **Lock consultivi (advisory):** il sistema operativo offre il lock ma non regola l'accesso al file: sono i processi che devono evitare di accedere al file se non hanno il lock.

I sistemi Windows adottano lock obbligatori, quelli Unix-like consultivi.

Tipi di file:

- Dati (numerici, testo, binari)
- Programmi

Il sistema operativo può essere più o meno consapevole del tipo di file, ma deve almeno riconoscere il tipo di file eseguibile.

Come riconoscere un file:

- Schema nome (nome.estensione).
- Attributi nel file.

Struttura di un file:

- **Nessuna struttura:** nei sistemi Unix il file è una sequenza di byte.
- **Sequenza di record:** righe di testo o record binari, a struttura e lunghezza fissa o variabile.

Directory:

Una directory è un elenco di file presenti nel file system, è una tabella che di associare il nome di un file ai dati contenuti nel file stesso. Sia i file che le directory risiedono su disco: deve esservi almeno una directory nel file system.

Operazioni dei processi sulle directory:

- **Creazione:** di un file in una directory.
- **Cancellazione:** di un file in una directory.
- **Ridenominazione:** di un file o spostamento da una directory ad un'altra.
- **Elenco:** dei file in una directory.
- **Ricerca: di un file:** basata sul nome, o su uno schema di possibili nomi.
- **Attraversamento:** del file system, ad esempio per backup.

Struttura delle directory:

- **Ad un livello:** una sola directory per tutti i file, è una struttura molto semplice, tuttavia, ha degli svantaggi come la denominazione dei file e il raggruppamento.

- **A due livelli:** la directory principale contiene delle sottodirectory, che a loro volta contengono i file dell'utente. Utenti diversi possono dare lo stesso nome a file diversi, occorre quindi usare dei nomi di percorso (path name) per identificare un file univocamente.
- **Ad albero:** ogni directory contiene ricorsivamente files e altre directory, permette agli utenti di raggruppare i propri file, per semplificare l'accesso ad ogni programma è assegnata una directory corrente, dalla quale si possono specificare path relativi. Con la directory cancellata tutto il suo contenuto, oppure permetto di cancellare una directory solo se è vuota.
- **A grafo aciclico:** permette di avere più di un nome per lo stesso file (aliasing). Il problema principale di questa struttura è il fatto che l'attraversamento del file system risulta molto complicato.

Protezione:

Le informazioni in un file system devono essere preservate sia dai danni che possono occorrere alle memorie fisiche sia dagli accessi impropri da parte di chi non ha diritto.

Il metodo più diffuso per cercare di controllare in che modo l'accesso ai file e alle directory del nostro computer vengono regolati è il controllo delle liste degli accessi. A ogni file o directory nel file system viene associata una lista che specifica quali sono gli utenti che hanno il diritto di accedere la file o alla directory con i relativi permessi.

Il file system prima di ogni accesso controlla chi è l'utente proprietario del processo e verifica quali sono i suoi diritti controllando l'access control list del file.

Il principale svantaggio di questa tecnica risiede nel fatto che le access control list possono diventare estremamente lunghe quando gli utenti e i permessi sono molti.

Nei sistemi Unix-like si usa un metodo diverso, che prevede il raggruppamento degli utenti in classi, l'idea è quella di definire dei gruppi di utenti che vengono associati a dei file o a delle directory. Quindi un file o una directory non ha solo un utente proprietario ma anche un gruppo proprietario. Per il file si distinguono tre domini per cui si possono stabilire dei diritti.

- **Proprietario:** l'utente che possiede la directory.
- **Gruppo:** il gruppo di utenti che condivide il file/directory di cui può far parte anche il proprietario.
- **Pubblico:** tutti gli altri utenti.

Per ognuno di questi domini viene definito il tipo di diritti che si possono avere sul file:

- **Diritto di lettura:** r
- **Diritto di scrittura:** w
- **Diritto di esecuzione:** x

Pertanto, ci possono essere nove combinazioni diverse di diritti che vengono espressi con una lista di bit.

Tramite i comandi **CHOWN** e **CHGRP** si possono cambiare rispettivamente i gruppi e l'utente proprietario e cambiare i diritti di un file.

Volumi

Un volume è una partizione di un determinato dispositivo di memorizzazione secondaria che contiene un filesystem come, ad esempio, una directory al cui interno ci sono diversi file. Un volume di solito è contenuto in una partizione, su un solo dispositivo. Quando viene formattata una chiavetta USB, al suo interno viene creato un volume.

Il sistema operativo deve permettere di montare e smontare un volume all'interno dello spazio dei nomi del filesystem. L'operazione di montaggio di un volume consiste banalmente nel collegare una

qualsiasi unità di memorizzazione esterna al nostro computer. L'operazione analoga è lo smontaggio, in cui si espelle il dispositivo tramite una serie di passaggi che variano a seconda del sistema operativo utilizzato.

Tuttavia, si può effettuare anche un'operazione di montaggio manuale che prevede di fornire al sistema operativo l'identificazione del dispositivo fisso e la partizione dove risiede il volume. Successivamente è necessario specificare il luogo di montaggio, ossia la locazione all'interno della struttura della directory in cui vogliamo che la radice del volume che stiamo montando venga inserita.

Interfaccia e struttura del kernel:

Il sistema operativo deve proteggere sé stesso dai programmi applicativi; infatti, le applicazioni possono accedere alla memoria del kernel e modificarlo, ciò sarebbe problematico per tutto il sistema di elaborazione. Il kernel può difendersi dalle modifiche dei programmi applicativi tramite l'ausilio dell'hardware, la CPU può funzionare in **modalità utente** o in **modalità di sistema** impostando un opportuno bit di modalità esposto dal processore.

I programmi applicativi vengono eseguiti in modalità utente, mentre il kernel viene eseguito in modalità di sistema. Alcune istruzioni del processore sono considerate privilegiate, ossia eseguibili solo in modalità di sistema.

Supponiamo che un processore metta a disposizione delle istruzioni per accedere alle periferiche di input/output, per escludere la possibilità che i programmi applicativi accedano a tali periferiche vengono proibite le istruzioni di accesso in modalità utente.

Se un certo processo in modalità utente cerca di eseguire un'istruzione macchina, viene generata un'eccezione, il controllo viene trasferito al kernel che elimina il processo. In modalità utente non si può accedere a tutta la memoria fisica ma solo ad un range stabilito dal kernel per il processo in esecuzione.

Implementazione di chiamate di sistema ed API:

Il sistema operativo che esegue in modalità kernel ha a disposizione tutte le istruzioni macchina e l'accesso a tutta la memoria di sistema.

Il kernel può impostare il bit di modalità che permette di eseguire la transizione dalla modalità di sistema alla modalità utente, viceversa non si può passare da modalità utente a modalità di sistema. Per passare in modalità di sistema l'utente ha a disposizione delle istruzioni simili agli interrupt, queste istruzioni vengono impostate come i vettori di interruzioni dal kernel quando il sistema operativo viene avviato, allo stesso tempo il kernel imposta anche il vettore per l'indirizzo a cui saltare per passare dalla modalità utente a quella di sistema.

Una volta impostata questa istruzione macchina viene generata l'eccezione software che fa passare la CPU in modalità di sistema e trasferisce il controllo alla subroutine del kernel che è il punto di ingresso unico di tutte le chiamate di sistema.

Siccome è disponibile un unico punto di ingresso oltre ai parametri è necessario passare un numero che identifichi quale chiamata di sistema viene passata. La subroutine che viene invocata è chiamata **system call interface**.

Passaggio dei parametri alle chiamate di sistema:

L'invocazione delle chiamate di sistema passa per un'eccezione software, pertanto sorge un problema di performance siccome si ha un passaggio da modalità utente a modalità kernel, questo passaggio rallenta la chiamata della system call interface. Ci sono diversi modi per ottimizzare la system call interface, in particolare il passaggio dei parametri:

1. Il metodo più semplice è utilizzare i **registri del processore** per passare i parametri, è un metodo veloce che consente di non utilizzare la memoria centrale, lo svantaggio principale è rappresentato dal fatto che i registri sono pochi, pertanto, tale metodo risulta utile quando i parametri e i loro dati sono in piccola quantità.
2. Un altro metodo utilizza dei **blocchi di memoria** che vengono allocati in memoria centrale, nei quali vengono memorizzati i parametri, l'indirizzo del blocco di memoria viene passato attraverso i registri. Questo metodo viene utilizzato in combinazione con il primo metodo dai sistemi Linux.
3. Il terzo metodo prevede un **push dei parametri sullo stack** delle chiamate, la write è una funzione, in modalità utente ho uno stack sul quale è possibile eseguire un'operazione di push delle chiamate. È un sistema molto più lento rispetto a quello che utilizza i registri.

Il doppio stack è utilizzato per garantire la sicurezza e l'integrità del sistema. Quando viene effettuata una chiamata di sistema, i parametri vengono copiati dallo stack della modalità utente allo stack della modalità kernel. Questo passaggio avviene per garantire che il kernel abbia accesso ai parametri in modo sicuro, evitando che un processo utente possa alterare direttamente i dati del kernel. L'utilizzo di due stack distinti contribuisce anche a garantire l'isolamento tra la modalità utente e la modalità kernel. I dati sensibili del kernel sono protetti dallo spazio utente, riducendo il rischio di accessi indesiderati o manipolazioni.

Uso delle librerie dinamiche per le API:

Le librerie di API e le librerie standard di linguaggio sono realizzate come librerie dinamiche, se fossero statiche potrebbe cambiare l'implementazione delle API, perchè possono cambiare le chiamate di sistema. Se le librerie standard e le librerie di API fossero linkate staticamente si verificherebbe un problema di ricompilazione ogni qual volta le API e le librerie standard subissero una modifica.

Il fatto che le librerie e l'eseguibile possano essere linkati dinamicamente dipende dal cambiamento dell'ABI (**application binary interface**). L'ABI è l'insieme delle convenzioni attraverso le quali il codice binario dell'applicazione si interfaccia con il codice binario della libreria dinamica delle API. Queste condizioni (ABI) riguardano diversi parametri per esempio:

- Naming delle funzioni.
- L'ordine in cui i parametri vengono messi sullo stack delle chiamate.
- In che modo vengono passati i tipi di dati (standard big endian/little endian).

Tutti questi fattori determinano la portabilità degli eseguibili binari sulle varie piattaforme.

Per rendere portabili le applicazioni si può:

- Scrivere l'applicazione in un linguaggio portabile, l'eseguibile è il file sorgente in linguaggi come Python.
- Scrivere l'applicazione in un linguaggio con un ambiente a runtime portabile come Java, l'eseguibile in questo caso è il bytecode.
- Scrivere l'applicazione utilizzando il linguaggio con un compilatore portabile ed API standardizzate, in questo caso l'eseguibile è il file binario compilato e linkato.

Struttura del kernel

Il kernel è suddiviso in base a quali servizi offre:

- Gestione dei processi e dei thread.
- Comunicazione tra processi e sincronizzazione.

- Gestione della memoria.
- Gestione dell'I/O.
- File system.

Il kernel è un programma di dimensioni molto ampie e notevole complessità; deve operare con velocità elevata al fine di non rallentare l'esecuzione dei programmi applicativi. Un malfunzionamento del kernel potrebbe provocare il crash dell'intero sistema di elaborazione.

Ci sono diverse possibilità di architettura del kernel:

Struttura monolitica:

Il kernel è un unico singolo file binario statico che fornisce un elevato numero di funzionalità come:

- Scheduling CPU.
- File system.
- Gestione della memoria, swapping, memoria virtuale.
- Device drivers.

Vantaggi:

- Elevate prestazioni.
- Efficienza.

Svantaggi:

- Complessità.
- Fragilità ai bug, un bug nel device driver può causare un crash dell'intero kernel.
- Necessità di ricompilare il kernel nel caso vengano messe in commercio nuove periferiche.

Struttura a strati:

Il software è diviso in vari strati, lo strato più basso interagisce con l'hardware, l'ennesimo strato interagisce solo con lo strato sottostante.

L'approccio offre due vantaggi:

- Ogni strato può essere progettato e implementato indipendentemente dagli altri.
- È possibile verificare la correttezza di uno strato indipendentemente da quella degli altri.

Pochi sistemi operativi utilizzano questo approccio in quanto risulta difficile definire esattamente quali funzionalità deve avere uno strato, inoltre ogni strato introduce un overhead che peggiora le prestazioni.

Microkernel:

I principali problemi dei kernel monolitici sono la fragilità e la complessità, con il microkernel la complessità viene ridotta al minimo spostando la maggior parte dei servizi del kernel fuori dal sistema operativo in programmi di sistema. All'interno del kernel vengono mantenuti solo un insieme minimo di servizi indispensabili che servono per implementare i servizi nei programmi di sistema. Viene definito microkernel perché offre pochi servizi.

Vantaggi:

- La facilità di estensione del sistema operativo, posso aggiungere nuovi servizi senza modificare il kernel.

- Maggiore affidabilità, se un servizio dovesse andare in crash non manda in crash il kernel stesso.

Svantaggi:

- Overhead.
- Una minore velocità di esecuzione delle chiamate di sistema.

Struttura a moduli:

il kernel è strutturato in componenti dinamicamente caricabili che comunicano tra di loro tramite delle interfacce. Quando il kernel ha bisogno di offrire un determinato servizio, ad esempio determinati device driver, carica il driver che gli serve e una volta che il servizio non è più necessario il driver viene scaricato.

Pochi sistemi operativi implementano una struttura pura del kernel, ma combinano i diversi approcci in maniera complessa in modo tale da ottenere requisiti di affidabilità, sicurezza e prestazioni.

Politiche e meccanismi:

- **Una politica** dice quando un'operazione viene effettuata, descrive quando un processore viene sottratto ad un determinato processo per essere assegnato a uno differente.
- **Un meccanismo** spiega in che modo un'operazione viene effettuata, come è possibile sospendere l'esecuzione di un certo processo in modo tale che questa possa essere ripresa in un secondo momento.

Differenze tra politiche e meccanismi:

- Le politiche impattano sulle caratteristiche percepite del sistema di elaborazione, i meccanismi se sono sufficientemente rapidi no.
- I meccanismi sono più stabili delle politiche, quest'ultime spesso cambiano in funzione delle caratteristiche percepite che vogliamo che il sistema di elaborazione abbia. Ad esempio, se necessito di un sistema che lavora real time potrebbe essere utile utilizzare un certo tipo di scheduling.

Processi e thread – la struttura:

Multiprogrammazione e multitasking:

Sono due concetti importanti in funzione dei due obiettivi che hanno i sistemi operativi riguardo l'esecuzione dei processi:

- **Efficienza:** mantenere impegnata la CPU il maggior tempo possibile nell'esecuzione dei programmi.
- **Reattività:** la reattività è importante per quei programmi che devono reagire prontamente ai comandi dell'utente. La reattività consiste nel dare l'illusione che ogni processo progredisca continuamente nella propria esecuzione come se avesse una CPU dedicata.

Le tecniche che vengono utilizzate nei sistemi operativi per ottenere questi due requisiti sono la multiprogrammazione e multitasking.

- **Multiprogrammazione:** la multiprogrammazione è una tecnica utilizzata per ottenere efficienza, il suo obiettivo è impedire che un programma che non è in condizione di proseguire l'esecuzione mantenga la CPU.

La multiprogrammazione non è una tecnica rilevante per i sistemi puramente batch, ovvero quei sistemi in cui non c'è interazione diretta con l'utente.

- **Multitasking:** il multitasking è una tecnica utilizzata per ottenere reattività, il suo obiettivo è far sì che un programma interattivo possa reagire agli input utente in un tempo accettabile. (cioè che un programma interattivo venga bloccato da un programma non interattivo).

Come funziona la multiprogrammazione:

Il sistema operativo mantiene in memoria tutte le immagini dei processi che vuole eseguire, quando la CPU non è impegnata il sistema operativo seleziona un processo in esecuzione e gli assegna la CPU. Se un processo che ha la CPU non può più procedere con l'esecuzione, gli viene sottratta la CPU e viene assegnata ad un altro processo. Come risultato, se i processi sono in numero maggiore rispetto alle CPU, queste saranno impegnate nell'esecuzione di qualche processo per la maggior parte del tempo.

Come funziona il multitasking:

Il multitasking è un'estensione della multiprogrammazione utilizzata per i sistemi interattivi, l'idea è che la CPU venga sottratta periodicamente ai programmi in esecuzione e assegnata ad un altro programma non in esecuzione. In questo modo tutti i programmi progrediscono in maniera continuativa nella propria esecuzione, anziché solo nei momenti in cui il programma che detiene la CPU si mette in attesa.

Burst CPU e I/O:

L'obiettivo della multiprogrammazione è massimizzare l'utilizzo della CPU, gli algoritmi di scheduling sfruttano il fatto che di norma l'esecuzione di un processo è una sequenza di Burst.

Esistono due tipi di burst:

I Burst sono degli intervalli di tempo in cui il processo svolge delle operazioni in maniera omogenea.

- **Burst della CPU:** è un periodo di tempo il cui il processo usa solo la CPU svolgendo dei calcoli.
- **Burst dell'I/O:** Il processo smette di utilizzare la CPU e si mette in attesa del completamento di un'operazione di input/output.

Programmi I/O bound:

- Elevato numero di burst CPU brevi.
- Ridotto numero di burst CPU Lunghi.
- Tipici dei programmi interattivi.

Programmi di CPU bound:

- Elevato numero di burst di CPU lunghi.
- Ridotto numero di burst di CPU brevi.
- Tipici dei programmi batch.

Stato di un processo:

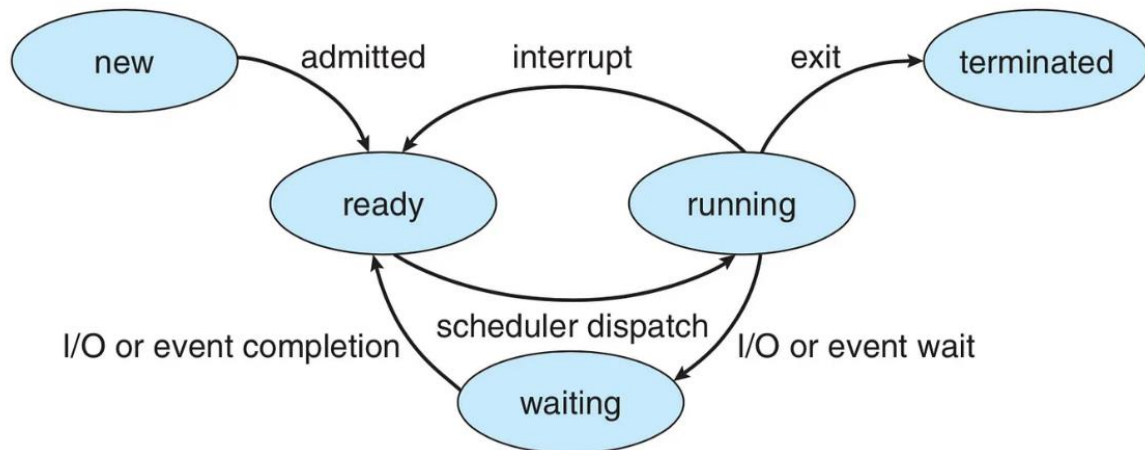
Durante l'esecuzione un processo cambia più volte stato, lo stato di un processo dipende da quali sono le sue condizioni di esecuzione.

I possibili stati di un processo sono cinque:

- **Nuovo:** il processo è creato ma non ancora ammesso all'esecuzione.

- **Pronto:** il processo può essere eseguito (è in attesa che gli sia assegnata una CPU).
- **In esecuzione:** le sue istruzioni vengono eseguite da qualche CPU.
- **In attesa:** il processo non può essere eseguito perché è in attesa che si verifichi qualche evento.
- **Terminato:** il processo ha terminato l'esecuzione.

Diagramma di transizione di stato dei processi:



Quando un processo viene creato entra nello stato **new**, creato ma non ancora ammesso a essere schedato, perché potrebbero già esserci troppi processi nello stato ready/running in modo tale da non creare un sovraccarico al sistema.

Se viene ammesso passa in stato ready con la transizione **admitted** ed è pronto per essere eseguito, quando riceve la CPU effettua la transizione di **scheduler dispatch** e passa in stato running, viene eseguito dalla CPU.

Con il multitasking periodicamente gli viene sottratta la CPU con un'operazione di **interrupt**, se il processo era in grado di continuare la sua esecuzione torna in stato ready, altrimenti si ha una transizione nello stato di **waiting**. Quando l'operazione di I/O termina la CPU fa passare il processo nello stato **ready**. Se il processo è in stato di running e invoca l'API opportuna per l'uscita **exit** il processo viene messo in stato **terminated** e ci rimane fino a che il sistema operativo non decide di riciclare le sue risorse.

Scheduler della CPU - scheduler a breve termine:

Lo scheduler della CPU è quel componente che sceglie tra tutti i processi in stato ready quelli che devono passare allo stato running ed alloca ad ognuno di essi un core libero. Se c'è un processore libero e un certo insieme di processi ready utilizza una politica per scegliere a quale processo dell'insieme assegnare il processore. Lo scheduler della CPU viene implementato con un insieme di code (code con priorità quindi non necessariamente FIFO). Sono delle strutture in cui i processi vengono messi in attesa e vengono ripescati un po' alla volta.

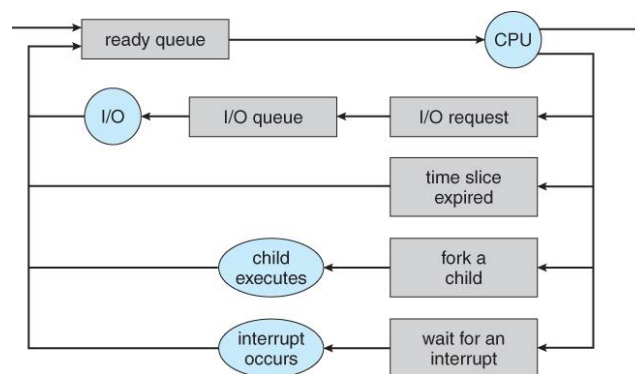
Esistono diverse politiche di scheduling:

- **Ready queue:** processi residenti in memoria e in stato ready.
- **Wait queue:** code per i processi che sono residenti in memoria e in stato wait; una coda diversa per ciascun diverso tipo di evento di attesa.

Generalmente c'è una sola ready queue, oppure in rari casi una ready queue per ogni processore. Ci sono invece tante wait queue per tutti i processi in stato wait, viene creata una wait queue per ogni possibile causa di attesa.

Quando un processo viene ammesso, cioè passa dallo stato new allo stato ready entra nella ready queue, lo scheduler della CPU pesca il **process control block**, il PCB è l'oggetto all'interno del kernel che rappresenta il processo, e gli dà il processore, a questo punto il processo diventa running. Il processo esegue per un determinato lasso temporale finché questo non scade, quando succede ciò il processore gli viene tolto e il processo viene reinserito nella ready queue. Prima o poi verrà ripescato e tronerà in esecuzione.

Se un processo avanza una richiesta per un input ma non può proseguire non avendo a disposizione una CPU si mette in attesa ed entra in una wait queue. Una volta finita l'operazione di input/output la routine di interrupt andrà nella coda I/O e risveglierà i processi che verranno riportati nella ready queue.



Schemi di scheduling:

Lo scheduler della CPU ha il compito di prendere la decisione di quale processo ready deve ricevere la CPU, le decisioni possono essere prese in quattro stati diversi:

1. Quando un processo passa da stato running a stato waiting.
2. Quando un processo passa da stato running a stato ready.
3. Quando un processo passa da stato waiting a stato ready.
4. Quando un processo termina.

Qualora il riassegnamento venga effettuato solo nelle situazioni 1 e 4 lo schema di scheduling è detto **nonpreemptive**, non si ha la sottrazione di un processore a un processo running. Altrimenti è detto **preemptive**, infatti il kernel libera un core sottraendolo forzatamente a un processo che lo sta utilizzando.

Per effettuare un riassegnamento preemptive è necessario invocare un componente chiamato **dispatcher** che implementa un meccanismo chiamato **cambio di contesto** (context switch), tale meccanismo congela lo stato del processo corrente, lo mette dentro al process control block, prende dal PCB i registri del nuovo processo e li mette nel processore.

Esempio:

Supponendo di avere solamente due processi p0 e p1, il dispatcher salva tutti i registri del processore nel PCB di p0, prende il PCB di p1 in cui saranno stati salvati tutti i registri quando p1 è stato bloccato, recupera tutti i registri e li rimette nel processore, incluso il program counter.

Latenza di dispatch:

La latenza di dispatch è il tempo impiegato dal dispatcher per fermare un processo e avviarne un altro, si tratta di tempo sprecato che è sottratto all'esecuzione dei programmi utente. Più il sistema operativo è complesso più il PCB è complesso e di conseguenza aumenta la latenza di dispatch.

Implementazione del multithreading:

I kernel degli attuali sistemi operativi sono strutturati usando un'architettura multithreading:

- **Thread a livello utente:** i thread del kernel vengono messi a disposizione degli utenti, previa un mapping tra i thread degli utenti e i thread del kernel, bisogna far lavorare i thread del kernel in modalità utente.
- **Thread a livello del kernel:** i thread implementati nativamente dal kernel, sono utilizzati per strutturare il kernel stesso in maniera concorrente.

Modelli di supporto multithreading:

I thread a livello del kernel vengono utilizzati dalle librerie di thread per implementare i thread a livello utente di un certo processo, a tale scopo possono essere adottate diverse strategie:

- **Molti a uno:** è un approccio di progettazione in cui molti thread utente vengono mappati su un singolo thread del kernel. In questo modello, l'intera gestione dei thread avviene nello spazio utente senza coinvolgere il kernel del sistema operativo per la gestione dei thread. Uno svantaggio di questa implementazione è il fatto che un thread a livello utente non è in grado di bloccare un altro thread a livello utente.
- **Uno a uno:** è un approccio di progettazione in cui ogni thread utente è mappato su un singolo thread del kernel. Con questa implementazione si verifica un problema sulle performance siccome il numero di thread da supportare diventa estremamente elevato perché i thread del kernel devono supportare anche i thread utente.
- **Molti a molti:** è un approccio di progettazione in cui tanti thread utente sono mappati su un certo numero di thread del kernel. Questo modello cerca di combinare i vantaggi dei due modelli precedenti, l'idea è quella di mappare un certo numero di thread utente su un minor numero di thread del kernel. Una variante del modello molti a molti è il **modello a due livelli** che implementa le funzionalità del modello molti a molti permettendo però anche di blindare dei thread uno a uno con un thread del kernel.

Criteri di valutazione algoritmi di scheduling:

Per valutare l'efficacia di un algoritmo di scheduling rispetto a un altro è necessario adottare misure specifiche. Tali metriche non dipendono solamente dalla natura dell'algoritmo, ma sono anche influenzate dalle caratteristiche del carico di lavoro considerato.

I principali criteri di valutazione sono:

1. **Utilizzo della CPU:** percentuale di tempo in cui la CPU è impegnata nell'esecuzione dei programmi utente.
2. **Throughput:** il numero di processi che completano l'esecuzione nell'unità di tempo.
3. **Tempo di completamento:** è il tempo necessario per completare l'esecuzione di un processo e dipende da vari fattori.
4. **Tempo di attesa:** è il tempo che un processo passa all'interno della ready queue, rappresenta un dato importante perché rappresenta tempo sprecato per l'assenza di una CPU.
5. **Tempo di risposta:** negli ambienti interattivi è definito come il tempo che un processo impiega dall'arrivo del primo input alla creazione di un output.

Principali algoritmi di scheduling:

1. **Scheduling in ordine di arrivo (first come first served):** la CPU viene assegnata al primo processo che la richiede. L'efficienza è basata sul processo che arriva per primo, se arriva prima un processo rapido con un burst di CPU molto corto, il tempo di attesa è più corto, questo fenomeno si chiama **effetto convoglio**.
2. **Scheduling per brevità (shortest job first):** la CPU viene assegnata al processo che ha il successivo burst di CPU più breve. Un vantaggio di questa implementazione è il fatto di avere un tempo di attesa minimo in quanto il meccanismo è FCFS. Lo scheduling per brevità si può usare quando i processi arrivano tutti nello stesso istante. Tenendo conto dei tempi di arrivo c'è una variante dell'algoritmo che prende il nome di **shortest-remaining time first**, utilizza la prelazione per gestire il caso in cui i processi non arrivino tutti nello stesso istante, se nella ready queue arriva un processo con un burst più corto di quello running, si effettua un cambio di contesto e si mette in esecuzione quello con il burst più corto.
3. **Scheduling circolare (round-robin):** funziona stabilendo a priori, un intervallo di tempo, detto **quanto di tempo** che dura dai 10 ai 100 millisecondi. Ogni processo che viene messo nella ready queue esegue per il quanto di tempo, una volta trascorso il quanto di tempo se il processo non ha esaurito il suo burst di CPU viene sospeso, viene riportato in fondo alla ready queue e si passa al processo successivo. La ready queue viene implementata come un buffer circolare, si inserisce un interrupt per il timer che scandisce il quanto di tempo.

Caratteristiche:

Supponiamo di avere n processi nella ready queue e un quanto temporale q , nessun processo per ottenere nuovamente la CPU aspetta più di $q(n-1)$. Ogni processo ottiene una percentuale perfettamente equa di CPU pari a $1/n$. Se q è lungo, può succedere che il burst è più corto del quanto temporale, se ciò accade per tutti i burst l'algoritmo degenera nel FCFS.

Se q è troppo corto e diventa paragonabile alla latenza di dispatch, allora fa crollare l'utilizzo della CPU. Il quanto di tempo deve essere sufficientemente lungo, rispetto alle latenze ma deve essere decisamente più piccolo della lunghezza dei burst di CPU.

L'algoritmo round robin va bene per carichi di lavoro interattivi (IO driven) con burst piccoli.

3. Scheduling con priorità:

Nello scheduling con priorità ad ogni processo viene assegnato un livello di priorità identificata con un numero. Ci sono due standard diversi, Unix e Windows, **in Unix** più è basso il numero più elevata è la priorità, **in Windows** più è alto il numero più è alta la priorità. Essendoci processi con priorità diverse viene eseguito sempre il processo con priorità più alta.

Lo scheduling con priorità può essere preemptive o non preemptive e può ammettere processi con lo stesso numero di priorità oppure no. Per fare lo scheduling di processi che hanno la stessa priorità si usa un altro algoritmo, **generalmente si utilizza round robin**.

L'algoritmo di scheduling con priorità, per determinati carichi, non garantisce l'esecuzione di tutti i processi, per esempio un processo con priorità molto bassa potrebbe non venir mai schedato, il problema dell'attesa indefinita è chiamato **starvation**.

Un modo per modificare l'algoritmo con priorità e ovviare al problema della starvation è quello di introdurre una variante chiamata **priorità con invecchiamento** che prevede l'introduzione di un quanto di tempo per far sì che a un processo con una certa priorità, trascorso un periodo di tempo con la stessa priorità, veda la sua priorità aumentata.

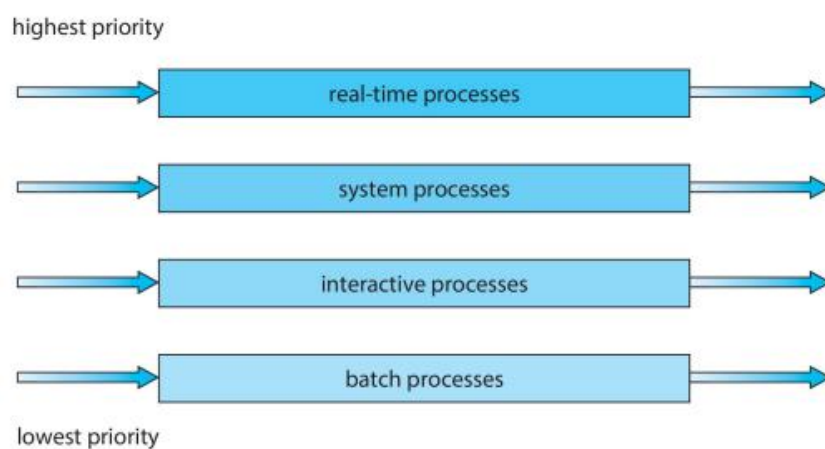
Code multilivello:

Per unire tanti algoritmi diversi e fare uno scheduler vengono utilizzate le code multilivello. L'algoritmo con priorità viene implementato utilizzando più di una ready queue, l'idea è quella di avere una ready queue per ogni livello di priorità.

Per capire qual è il prossimo processo lo scheduler parte dalla coda a priorità più alta, se non è vuota esegue il processo e lo rimette in fondo. Il meccanismo di priorità viene utilizzato per dare un funzionamento diverso a seconda del tipo di processo, utilizzo le priorità per distinguere tra di loro i processi a seconda della loro tipologia, i processi che devono reagire più velocemente all'I/O hanno una priorità più alta, i processi che sono più CPU bound sono a priorità più bassa.

Code multilivello con retroazione:

La priorità di un processo varia dinamicamente, la priorità di un processo viene abbassata se questo è CPU bound, ovvero se ha dei burst di CPU lunghi. Viceversa, faccio migrare verso l'alto il processo basandomi sull'aging, aumento la priorità se non riesco a schedulare il processo.



Gestione della memoria - la struttura:

Il problema di allocazione della memoria per i sistemi operativi consiste nel fatto che il sistema operativo deve allocare diverse zone di memoria centrale ai diversi processi, questo problema si suddivide in ulteriori tre sotto problemi.

1. Che strategia di allocazione è possibile adottare e come è possibile dividere e assegnare lo spazio della memoria centrale.
2. Come si può proteggere la memoria data ad un processo dagli accessi indesiderati fatti da altri processi.
3. Rilocazione, il programma deve referenziare le zone di memoria che ho allocato.

Strategia di allocazione contigua:

La memoria centrale è partizionata in due zone, una zona viene adibita al sistema operativo, in particolare al kernel e un'altra ai processi utente. Ogni processo utente occupa per la propria immagine una zona contigua di memoria che parte da un indirizzo e arriva ad un altro per tutto l'intervallo compreso.

Protezione con registro base e limite:

La protezione della memoria viene eseguita tramite hardware, sono necessarie delle modifiche al processore in modo tale che questo fornisca delle funzionalità di supporto al sistema operativo. Si aggiungono al processore due registri, un registro che viene detto base e un secondo registro che viene detto limite.

I registri base e limite possono essere impostati solo in modalità di sistema, in modalità utente invece vengono utilizzati per filtrare gli indirizzi che vanno dal processore alla memoria.

Prima che un indirizzo arrivi alla memoria viene controllato che sia maggiore o uguale della base, la base rappresenta l'indirizzo di memoria più basso ammissibile. Se l'indirizzo è sotto la base viene scartato. Successivamente si effettua un altro controllo, se l'indirizzo è minore della somma base + limite viene scartato. Il limite indica la dimensione della regione contigua.

Se entrambi i controlli passano, l'istruzione macchina viene eseguita. La soluzione con registro base e limite non fornisce uno spazio di indirizzamento virtuale ai processi poiché inserisce in memoria lo stesso indirizzo che arriva dal bus, l'unico modo per implementare uno spazio di indirizzamento virtuale è utilizzare un binding in fase di esecuzione.

MMU con registro di rilocalizzazione

La soluzione con MMU implementa il binding in fase di esecuzione, il compilatore può compilare un programma in C come se il programma fosse caricato sempre a partire dall'indirizzo 0. L'idea è che il processore produca degli indirizzi logici che vengono trasformati da un'unità detta **MMU** che effettua la rilocalizzazione e traduce gli indirizzi logici in indirizzi fisici.

Il binding in fase di esecuzione è l'unico metodo in cui gli indirizzi logici e fisici differiscono, in quanto ho nel mezzo una unità hardware che li traduce.

La MMU più semplice con schema di allocazione contigua è la **MMU con registro di rilocalizzazione**, è una variazione dello schema con registri base e limite, il registro base diventa un registro di rilocalizzazione, l'indirizzo fisico viene ottenuto sommando all'indirizzo logico il valore del registro di rilocalizzazione.

Il sistema operativo imposta il registro di rilocalizzazione con il valore base corrispondente al segmento o alla pagina attualmente in esecuzione per il processo. Quando il processo genera un indirizzo virtuale, l'MMU somma questo indirizzo al valore contenuto nel registro di rilocalizzazione.

La somma dell'indirizzo virtuale e del valore nel registro di rilocalizzazione produce l'indirizzo fisico corrispondente nella memoria, ovvero la posizione effettiva in cui i dati possono essere letti o scritti.

Svantaggi dell'allocazione contigua:

Le partizioni della memoria centrale vengono dimensionate in base alla necessità del processo. Si crea una situazione di frammentazione esterna, in cui ho abbastanza memoria per mettere l'immagine di un nuovo processo ma non contigua. Per ovviare a questo problema si può effettuare una deframmentazione della memoria.

Allocazione contigua a partizioni variabili:

Il kernel deve mantenere una lista di tutti i buchi disponibili, da dove inizia e dove finisce la zona di memoria di ogni buco. Alla creazione di un nuovo processo il sistema operativo deve scegliere un buco all'interno del quale inserire l'immagine del processo.

Ci sono diverse strategie per decidere quale sia la locazione di memoria migliore:

le tre strategie più comuni sono:

- **First-fit:** si sceglie il primo buco sufficientemente grande da contenere l'immagine del processo, è anche la procedura più rapida.
- **Best-fit:** si sceglie il buco più piccolo che riesce a contenere l'immagine del processo in modo tale da ridurre lo spreco di memoria.
- **Worst-fit:** si sceglie il buco più grande in modo tale da avanzare più spazio, tale spazio potrebbe essere sufficientemente grande da contenere un'altra immagine.

Nessuna delle tre tecniche è migliore di un'altra, tuttavia first-fit risulta la più rapida, mentre best-fit funziona meglio.

Frammentazione esterna:

La memoria libera è sufficiente per la creazione di un nuovo processo ma è sparsa tra buchi non contigui e troppo piccoli. Questo problema è tipico dello schema di allocazione contigua a partizioni variabili.

Frammentazione interna:

Si verifica la presenza di frammentazione interna quando si utilizzano partizioni fisse e la quantità di memoria allocata a un processo supera la quantità di memoria necessaria. In questo contesto, una partizione può contenere porzioni di memoria non utilizzate, generando così la frammentazione interna.

Nell'allocazione contigua a partizioni variabili lo spazio inutilizzabile tende ad essere circa il 50% dello spazio utilizzato. Ad esempio, se i processi utilizzano 2MB di memoria c'è 1 mega inutilizzato per colpa della frammentazione, quindi 1/3 della memoria del computer va sprecata per colpa della frammentazione.

Una soluzione è la compattazione che prevede lo spostamento delle partizioni in modo tale da avere degli slot unici, questa tecnica tuttavia è molto onerosa e nel caso un processo stia effettuando delle operazioni di input/output non può essere spostato nella memoria.

Paginazione:

La paginazione consiste nel permettere una locazione di memoria ai processi che non sia contigua. Per ricostruire l'immagine del processo che è frammentata in diverse zone di memoria è necessario utilizzare l'hardware, in particolare una MMU più avanzata. La memoria centrale viene divisa in **frames**, blocchi di dimensione fissa, la dimensione dei frames dipende dall'architettura dell'hardware. Lo spazio di indirizzamento virtuale dei processi è diviso in pagine che hanno la stessa dimensione dei frames.

Dentro la MMU c'è un meccanismo che fa un mapping fra le pagine di un processo e i frame della memoria, a tale scopo si utilizza una tabella detta **tabella delle pagine**.

La tabella delle pagine associa le pagine di un processo ai frame in memoria corrispondenti, in questo modo la MMU leggendo la tabella delle pagine, traduce gli indirizzi logici in indirizzi fisici. Il vantaggio è l'assenza della frammentazione esterna e una totale indipendenza tra indirizzi logici e fisici, tuttavia c'è frammentazione interna.

La tabella delle pagine è un elenco, ha tante righe quante sono le pagine di un processo; quindi, ogni processo ha una sua tabella delle pagine. Una riga della tabella delle pagine deve contenere i frame della pagina corrispondente.

Traduzione degli indirizzi logici:

L'indirizzo logico viene diviso in due parti, numero di pagina e offset di pagina. Supponendo di avere un offset di 10 bit e un indirizzo di 32 bit, il numero di pagina è dato dai restanti bit ovvero 22. Nello spazio di indirizzamento virtuale del processo ci sono 2^{22} pagine, ogni pagina è della dimensione di 2^{10} byte quindi 1 Kbyte.

Frammentazione interna e dimensione di pagina:

Il vantaggio di questa implementazione è sostituire il problema di partizione esterna con quello di partizione interna, con la frammentazione interna assegno al processo una partizione più grande che non viene utilizzata interamente. La frammentazione interna media di un processo in generale spreca un pezzo dell'ultima pagina, quindi circa mezzo frame per processo.

Supporto alla paginazione nel sistema operativo:

Il sistema operativo deve tracciare in memoria la posizione della tabella delle pagine di ciascun processo, inoltre deve mantenere una tabella dei frame per ricordarsi quali sono i frame della memoria centrale che sono liberi rispetto a quelli che sono occupati.

Supporto hardware alla paginazione:

La tabella delle pagine viene mantenuta in memoria, la MMU ha due registri:

- **Page table base register:** indirizzo in memoria di dove inizia la tabella delle pagine.
- **Page table length register:** contiene la lunghezza della tabella delle pagine.

Il problema della paginazione è il fatto che rallenta gli accessi in memoria, ogni qualvolta si vuole accedere alla memoria è necessario effettuare due accessi, uno per accedere alla tabella delle pagine e tradurre l'indirizzo logico in indirizzo fisico, un altro per mettere l'indirizzo sul bus e accedere al dato. Il tempo di accesso in memoria viene raddoppiato pertanto si utilizza una cache formata da alcune righe della tabella delle pagine all'interno della MMU, tale cache è denominata **translation lookaside buffer (TLB)**, è una memoria di tipo associativo che mantiene al suo interno sia il numero di pagina che il numero di frame.

Il principale problema del TLB consiste nel fatto che ogni volta che viene effettuato un cambio di contesto è necessario svuotare completamente il TLB e inserire le entries di un altro processo, la cache deve essere svuotata completamente per poi essere riempita nuovamente. Pertanto, si arricchisce la cache con un **address space identifier**, si aggiunge a ogni riga del TLB un numero identificativo del processo, in questo modo si possono mantenere le entries in più tabelle delle pagine.

Il tempo effettivo di accesso alla memoria con paginazione dipende dal tasso di accesso al TLB. Si verifica un TLB hit se un certo numero di pagina di un indirizzo logico si trova nel TBL, altrimenti ho un TLB miss.

Politiche di sostituzione del TLB:

Se capita un TLB miss ma il TLB è pieno potrebbe essere una buona idea effettuare una sostituzione di un entry, perchè nell'immediato futuro questa potrebbe essere richiesta.

Vengono usate diverse strategie come:

- Last recently used.
- round robin.
- Casuale.

Alcuni processori non implementano alcuna di queste strategie, in maniera tale da permettere al sistema operativo di partecipare alla decisione di quale entry sostituire.

Protezione della memoria:

Piuttosto che inserire il registro PTLR, che permette di tagliare la tabella delle pagine in maniera da ridurre gli indirizzi disponibili al processo si può utilizzare un bit di validità che indica la validità o meno della entry. Si possono inserire altri bit all'interno della tabella delle pagine come il bit di protezione che indica se la tabella è read-only o read-write e il bit di esecuzione per vedere se è eseguibile o no.

Condivisione della memoria:

La paginazione permette di implementare la memoria condivisa, è sufficiente che uno stesso frame di memoria fisica sia messo nella tabella delle pagine di più processi.

Strutture delle tabelle delle pagine per spazi di indirizzamento virtuali:

Quando i processori hanno degli spazi di indirizzamento virtuali molto grandi, per esempio, di (32 o 64) bit le tabelle delle pagine possono diventare a loro volta molto grandi. Per tale motivo esistono varie soluzioni per strutturare le tabelle delle pagine in maniera che siano sufficientemente piccole/efficienti:

- **Tabella delle pagine gerarchiche:**

L'idea è quella di paginare una tabella delle pagine, non allocare la tabella delle pagine in una zona di memoria contigua. Ogni numero di pagina è diviso a sua volta in un numero di pagina e in un offset. Un vantaggio delle tabelle delle pagine gerarchiche è il fatto che permette di supportare tabelle delle pagine di dimensioni diverse. Il principale svantaggio di avere una tabella delle pagine gerarchica è il fatto di aumentare il numero di accessi alla memoria, nel caso pessimo occorrono tre accessi alla memoria.

- **Tabella delle pagine di tipo hash:**

L'idea è di fare una tabella delle pagine più piccola di quella normale e di utilizzare una funzione hash implementata in hardware per calcolare un hash del numero di pagina e utilizzare quello come numero di riga all'interno della tabella. Se ci sono delle collisioni la tabella deve avere una lista concatenata di elementi per gestirla.

- **Tabella delle pagine invertite:**

Può essere anche chiamata tabella dei frame, anziché fare una tabella che mappa le pagine sui frame si effettua l'operazione inversa, ovvero una tabella che mappa i frame sulle pagine.

La tabella per ogni riga avrà il numero di frame della memoria fisica e per ogni frame viene riportato il numero di pagina del processo con l'ASID annesso. I vantaggi consistono nel fatto che non è più necessario avere una tabella per processo, basta un'unica tabella per memoria. Mentre nella tabella normale si accede in modo diretto, in questo caso devo effettuare una ricerca della riga che contiene l'ASID e il numero di pagina del mio processo. Non è possibile fare la condivisione della memoria tra processi diversi.

Swapping:

Generalmente è possibile eseguire un processo solo se tutta la sua immagine si trova in memoria, se ho tanti processi questi potrebbero chiedere così tanta memoria che andrebbe a superare la quantità di memoria disponibile. Se la memoria è poca rispetto al numero dei processi, pochi di questi possono essere ammessi all'esecuzione. L'idea dello swapping è prendere l'immagine di un processo e copiarla su memoria di massa (backing store), ciò permette di sospendere il processo e consente ad un altro processo di andare in memoria.

Ci sono due varianti dello swapping:

- **Standard:** consiste nel prendere tutta l'immagine di un processo e copiarla completamente sulla backing store, prendere un'altra immagine di un processo nella backing store e copiarla in memoria. Queste due operazioni vengono dette di (swap-out) e (swap-in).
- **Con paginazione:** si effettua solo uno (swap-in/swap-out) parziale, quando viene sospeso un processo, anziché scrivere tutta la sua immagine nella backing store vengono liberate un po' di pagine facendo l'operazione di page out, in modo tale da liberare uno spazio di memoria e fare page-in delle pagine del processo sospeso.

Grado di multiprogrammazione e utilizzo dei processori:

Il grado di multiprogrammazione è il numero massimo di processi che lo scheduler della CPU può mandare in esecuzione e determina l'occupazione totale di memoria fisica da parte dei processi.

Il grado di multiprogrammazione è pari al numero di immagini in memoria, in generale si chiamano processi in memoria quelli che sono nello stato ready-running-waiting e processi fuori memoria tutti quelli sospesi e messi nella backing store. Il grado di multiprogrammazione è legato all'utilizzo della CPU, se il grado di multiprogrammazione è basso anche l'utilizzo della CPU sarà basso, viceversa se aumento il grado di multiprogrammazione aumenta il numero di processi ready e quindi aumenta anche l'utilizzo della CPU.

Controllo del grado di multiprogrammazione:

- **Scheduler a lungo termine (admission scheduler):** è lo scheduler che decide se ammettere un processo dopo che questo è stato creato, in questo modo influisce sul grado di multiprogrammazione sul lungo periodo.
- **Scheduler a medio termine:** è lo scheduler che decide se un processo deve essere sospeso o meno e portare in memoria un altro processo.
- **Scheduler a breve termine:** non influisce sul grado di multiprogrammazione.

Memoria virtuale:

È la tecnica che permette di eseguire un processo senza avere tutta la sua immagine in memoria, permette di poter eseguire un processo che ha un'immagine molto più grande della memoria hardware a disposizione. Dal momento che raramente la memoria di un processo viene utilizzata tutta e soprattutto nello stesso momento, eseguire un programma anche se la sua memoria è caricata solo in parte, aumenta il grado di multiprogrammazione, è possibile portare in memoria molti più processi.

La memoria virtuale può essere implementata tramite:

- Paginazione su richiesta.
- Segmentazione su richiesta.

La paginazione su richiesta prevede di caricare in memoria una pagina solo quando viene utilizzata e il resto dell'immagine del processo risiede nella backing store. Questa tecnica è simile allo swapping con paginazione, tuttavia lo swapping individua le pagine da scaricare/caricare in maniera predittiva mentre la paginazione su richiesta lo fa in base all'uso.

La paginazione su richiesta deve essere supportata dall'hardware di paginazione, inoltre la tabella delle pagine deve possedere il bit di validità per indicare se una certa pagina non è valida o è assente dalla memoria. Se un programma tenta l'accesso a una pagina che ha il bit di validità impostato a zero la MMU genera una interruzione di Page fault e porta la pagina in memoria.

Gestione di un page fault:

Supponiamo che un processo esegua una certa istruzione macchina, verifica il bit di validità nella tabella delle pagine, se invalido la pagina non esiste in memoria. A questo punto il sistema operativo genera un interrupt di page fault, il controllo viene passato al kernel che mette il processo in attesa, cerca un frame libero nella lista dei frame, cerca nella backing store la pagina e schedula l'operazione di caricamento della pagina dalla backing store. Dal momento che tale operazione è lenta si potrebbe verificare un cambio di contesto. Viene caricata la pagina all'interno del frame libero, quando l'operazione è terminata il kernel cambia la tabella delle pagine del processo inserendo al suo interno il valore del nuovo frame e impostando il valore del bit di validità a valido, a questo punto il processo viene messo in stato ready.

Politiche di caricamento delle pagine:

- **Paginazione su richiesta pura:** le pagine vengono caricate solo quando servono, all'avvio di un processo non viene caricata alcuna pagina in memoria.

- **Prefetching:** vengono caricate altre pagine in un intorno della pagina richiesta.